

Review

ERICSSON
TECHNOLOGY

SPOTLIGHT ON
**ARTIFICIAL
INTELLIGENCE**



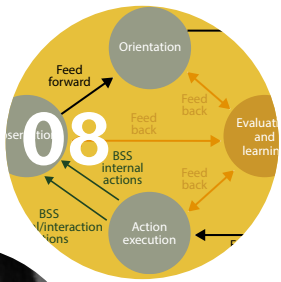
ERICSSON





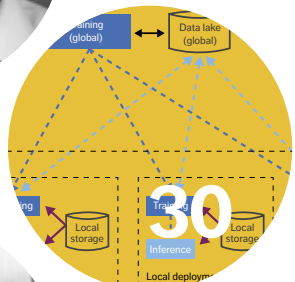
08 BSS AND ARTIFICIAL INTELLIGENCE – TIME TO GO NATIVE

The growing need to support disruptive services emerging from the Internet of Things and 5G requires a fundamental transformation of business support systems (BSS). At Ericsson, we believe that the best way to achieve this is by forging BSS and artificial intelligence (AI) together to create truly AI-native BSS.



18 OPEN, INTELLIGENT AND MODEL-DRIVEN: EVOLVING OSS

Ericsson is leveraging the open-source approach to build open and intelligent operations support systems (OSS) that are designed to support autonomic networks and agile services. Our concept benefits from our ability to combine the open-source approach with our unique network and domain knowledge.

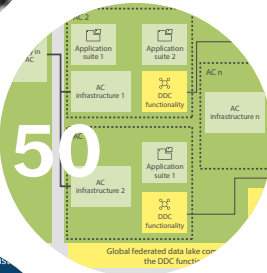


30 PRIVACY-AWARE MACHINE LEARNING WITH LOW NETWORK FOOTPRINT

Federated learning makes it possible to train machine learning models without transferring potentially sensitive user data from devices or local deployments to a central server. As such, it addresses security and privacy concerns at the same time that it improves functionality and performance.

40 COGNITIVE PROCESSES FOR ADAPTIVE INTENT-BASED NETWORKING

Extensive automation will be necessary to cope with the unprecedented flexibility and dynamic adaptation that 5G networks are introducing into service delivery and network resource utilization. Our intent-based networking approach combines several AI techniques within a flexible architecture and uses intents to specify what the autonomous system is expected to do.

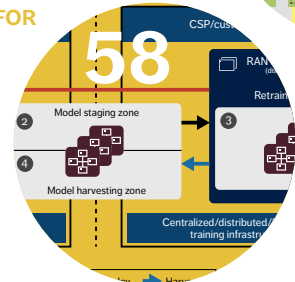


50 DATA INGESTION ARCHITECTURE FOR TELECOM APPLICATIONS

Our harmonized data ingestion architecture is designed to help communication service providers ensure superior performance across their operations, including everything from network management to product serviceability and tailored service offerings. An additional benefit of our approach is that it frees up application development resources to focus on use-case realizations instead of data management.

58 ARTIFICIAL INTELLIGENCE IN RAN – A SOFTWARE FRAMEWORK FOR AI-DRIVEN RAN AUTOMATION

When used correctly, AI techniques have tremendous potential to overcome complex cross-domain automation challenges in radio networks. Ericsson has identified the key software enablers for AI-based RAN automation and integrated them into a comprehensive framework that provides a solid and flexible technological foundation.



Ericsson Technology Review brings you insights into some of the key emerging innovations that are shaping the future of ICT. Our aim is to encourage an open discussion about the potential, practicalities, and benefits of a wide range of technical developments, and provide insight into what the future has to offer.

ADDRESS

Ericsson
SE -164 83 Stockholm, Sweden
Phone: +46 8 7190000

PUBLISHING

All material and articles are published on the Ericsson Technology Review website:
www.ericsson.com/ericsson-technology-review

PUBLISHER

Erik Ekudden

EDITOR

Tanis Bestland (Nordic Morning)

EDITORIAL BOARD

Magnus Buhrgard, Magnus Ewerbring,
Dan Fahrman, John Fornehed,
Kjell Gustafsson, Jonas Högborg,
Johan Lundsjö, Cecilia Nyström,
Håkan Olofsson, Patrik Roseen,
Anders Rosengren, Robert Skog,
Ben Wilmot and Sara Kullman

ART DIRECTOR

Carola Pilarz (Nordic Morning)

PROJECT MANAGER

Susanna O'Grady (Nordic Morning)

LAYOUT

Carola Pilarz (Nordic Morning)

ILLUSTRATIONS

Jenny Andersén (Nordic Morning)

SUBEDITORS

Ian Nicholson (Nordic Morning)
Paul Eade (Nordic Morning)

ISSN: 0014-0171

Volume: 104, 2021

BUILDING TRUST IN TELECOM AI FOR THE BENEFIT OF ALL

■ **ARTIFICIAL INTELLIGENCE (AI)** is without question the primary enabling technology for the next generation of system automation. Aside from freeing up valuable resources, improving performance and boosting innovation, AI also opens up a wealth of opportunities for communication service providers (CSPs) to grow their businesses beyond simply providing connectivity.

One of the biggest challenges to the wide-scale adoption of AI, however, is the need to fully address the trust issues related to the technology, particularly with regard to cultural and educational differences. At Ericsson, we believe that the best way to build trust is through knowledge sharing and dialogue with stakeholders about the specific AI techniques that we are exploring, testing and using in our products and solutions. The critical infrastructure that we provide for digital enterprises makes it imperative that our customers are confident that we are deploying AI in a trustworthy way in networks, using techniques that are robust, explainable, traceable and unbiased.

The purpose of this special issue of Ericsson Technology Review is to offer our stakeholders greater insight into our view of the role of AI in future network development. In it, we present our latest findings about the potential for trustworthy AI techniques to help us make breakthroughs in six key areas: business support systems (BSS), operation support systems (OSS), privacy, data ingestion, RAN and overall network performance.

BSS and OSS both have pivotal roles to play in the future development of our industry, particularly when it comes to 5G and the Internet of Things (IoT). Many 5G and IoT use cases require BSS that can handle complex business situations and optimize outcomes with minimal manual

...WE ARE DEPLOYING AI IN A TRUSTWORTHY WAY IN NETWORKS...

intervention. AI-native BSS enables the various applications within the BSS to share business information with each other in an efficient and secure manner. Similarly, our AI-powered OSS concept, which enables the use of industry-defined interfaces and open-source modules, offers big advantages over traditional OSS – especially when it comes to supporting emerging 5G and IoT use cases.

Advanced AI techniques are also proving to be enormously helpful in addressing privacy issues and reducing network footprint. Our research indicates that migration from a conventional machine learning model to a federated learning model – a more advanced AI technique – dramatically reduces the amount of information that is exchanged between different parts of the network with no negative impact on QoE.

AI techniques also open up new possibilities to achieve the level of RAN automation required to solve the complex resource management challenges presented by 5G and future radio systems. For example, our RAN experts have already integrated the key software enablers for AI-based RAN automation into a comprehensive framework that provides a solid and flexible technological foundation for the development of AI-based RAN. To help CSPs ensure superior performance across their operations, we have also used AI techniques to develop a harmonized data ingestion architecture that ensures secure and efficient access to data.

Reaching our ultimate goal of creating fully cognitive, zero-touch networks will require AI that is on par with human capability to reason and decide within complex dependencies and across broad domains. While AI is not yet able to offer that level of

sophistication, a group of Ericsson experts have proven that it is already possible to reach a high degree of practical autonomous operation in networks by combining existing AI techniques within a flexible architecture to create what we call a cognitive layer. It's an exciting step in the right direction.

We hope this special issue of our magazine inspires you to think about the role AI-based solutions will play in your organization in the years ahead. Please share it with your colleagues and business partners to help us spread the word about the benefits that trustworthy AI will bring to our industry. You can find both PDF and HTML versions of all the articles at: www.ericsson.com/ericsson-technology-review



ERIK EKUDDEN

SENIOR VICE PRESIDENT,
CHIEF TECHNOLOGY OFFICER AND
HEAD OF GROUP FUNCTION TECHNOLOGY

BSS and artificial intelligence

–TIME TO GO NATIVE

The growing need to support disruptive services emerging from the Internet of Things (IoT) and 5G requires a fundamental transformation of business support systems (BSS). At Ericsson, we believe that the best way to achieve this is by forging BSS and artificial intelligence (AI) together to create truly AI-native BSS.

LARS ANGELIN,
JOHAN SILVANDER

Although AI is of obvious benefit in terms of business optimization, and has been used in all sorts of businesses for decades, AI and BSS have never been integrated into one efficient system.

■ Examples of areas in which AI is already used in conjunction with BSS software include customer retention, chatbots, revenue and cost predictions, customer analysis, customer experience management, customer yield optimization, automation, process reengineering, simulations, quality improvements, and fraud and anomaly detection.

AI capabilities enable improved business decision dynamics and better decision precision, resulting in better business performance and agility.

Virtually all business activities can and will benefit from AI, and as 5G and the IoT continue to expand, the number of use cases will only continue to grow. The challenge we face at present is that the learning, insight-building and reasoning capabilities of AI in today's telco BSS are not as strong as they need to be to cope with emerging use cases.

For the most part, AI capabilities today are simply bolted onto telco BSS one by one. But this is inefficient in terms of life-cycle costs, because the BSS must be repeatedly upgraded to benefit from the AI algorithms. Further, as they are separate systems, the BSS information must be transformed to fit AI systems, and vice versa. A much more efficient alternative is AI-native BSS – that is, BSS with intrinsic AI capabilities where the AI logic

is a natural part of BSS logic in terms of both design and operation. This approach results in a system that can handle more complex business situations, generating more optimized business outcomes.

BSS evolution drivers

The main growth opportunities for communication service providers (CSPs) within the next decade are 5G and the IoT, with an estimated annual value of approximately USD 600 billion [1, 2]. To capitalize on this opportunity, CSPs must be able to support a marketplace with an ecosystem of many actors that have their own business models, where each actor may be both supplier and customer to other actors. Business support complexity increases dramatically in this environment. Current telco BSS, which can only support a single enterprise shop with a few business models, are simply not up to the task.

Surveys show that a clear majority of the telecommunications industry actors expect AI to have significant business impact in the coming five years, affecting both the top and bottom lines. They also expect AI to bring a significant competitive advantage to the enterprises using them, growing proportionally with AI usage. Analysts predict that enterprises will invest in AI competence, AI maturity and in organizational AI capabilities [3, 4], despite the costs [5].

ARTIFICIAL INTELLIGENCE

Artificial intelligence (AI) depends on software algorithms. At Ericsson, we use the term AI in its widest sense, including several subfields such as machine learning, representation learning and deep learning. AI-related areas such as natural-language processing, automated reasoning, multiagent systems, symbolic learning, knowledge representation, intelligent tutoring systems and high-level computer vision are also included [8].

There are essentially three main forces driving the combined AI-BSS evolution [1]. Firstly, business agility is a highly valued BSS property since the business itself evolves and new business opportunities emerge. AI plays a key role in both identifying opportunities and in shaping the new business models to pursue them. Secondly, the maturity of the communications industry is driving ever lower business transaction costs, as demonstrated by existing platform players like Amazon and Alibaba. In light of this, AI-supported process automation and reengineering are the tools

Terms and abbreviations

AI – Artificial Intelligence | BSS – Business Support Systems | CRISP-DM – Cross-Industry Process for Data Mining | CSP – Communication Service Provider | IoT – Internet of Things

●● A BUSINESS INTENT STATES THE DESIRED OR OPTIMAL OUTCOME OF A GIVEN SITUATION ●●

of choice. Finally, the cloud provides an ideal foundation for continuous introduction of AI and BSS marketplace capabilities. This is because the cloud offers deployment flexibility, elastic scaling and a micro-service architecture that enables a more fine-grained separation of concerns and componentization with loose coupling.

Key challenges to adding AI to BSS

Introducing AI into existing BSS is not straightforward. Some challenges, such as data acquisition, data piping and training algorithms, are obvious and well known. Others, however, are less so. One example of a less obvious challenge is the fact that the interpretation of the AI results requires business competence; another is that monetization requires both retraining within the organization and redesigning of the existing set of business rules and processes and system reengineering [5].

Traditional, non-AI-native BSS are divided into component silos – such as customer relationship management, catalogs, billing and order management – each with their own information. This arrangement contradicts AI efficiency and dynamics enablement, which require an open, pan-BSS information and rules view. When an AI capability is added to this environment, it is treated as an add-on, requiring both AI and BSS system competence, information transformation and in many cases partial system re-implementation or reconfiguration. BSS performance issues such as latency and scaling may arise.

Many business situations are multifaceted, have many root causes and may include both gains and risks. In many cases, a main business intent (also known as a KPI) must be broken down into a combination of subintents. These will be based on many data sets and algorithms, and then be stitched together by a super-algorithm to deliver the main business intent. There is also a risk of lost business control, as a high-level intent may affect many of the lower-level business rules and processes. This effect is considerably smaller when an intent is introduced at lower levels, but in those cases there will be less business gain. The uniqueness of an intent and its context means there is little opportunity for reuse or experience building.

Ericsson believes that a new BSS architecture style that incorporates AI-native properties – including data-centric, learning loop, intent and event-driven logic, business rule hierarchy, and support for strategic, tactical and operational levels – is a much more efficient way to integrate AI with BSS. We fully agree with the view that future BSS and AI will be inseparably linked and must mature together [4, 5].

Introducing intents to BSS

An enterprise is a hierarchical or line-of-command structure in which business rules at the top steer, align and control the activities and behaviors further down in the structure. Business rules also steer all behavior in BSS, in pursuit of the goal of creating and maintaining a successful business. The step between a business rule and a business intent is very small; just a small shift of perspective.

A business rule is static and states what to do in a given situation, while a business intent states the desired or optimal outcome of a given situation – that is, interpreting what the stakeholder’s interest is and trying to deliver as close to it as possible.

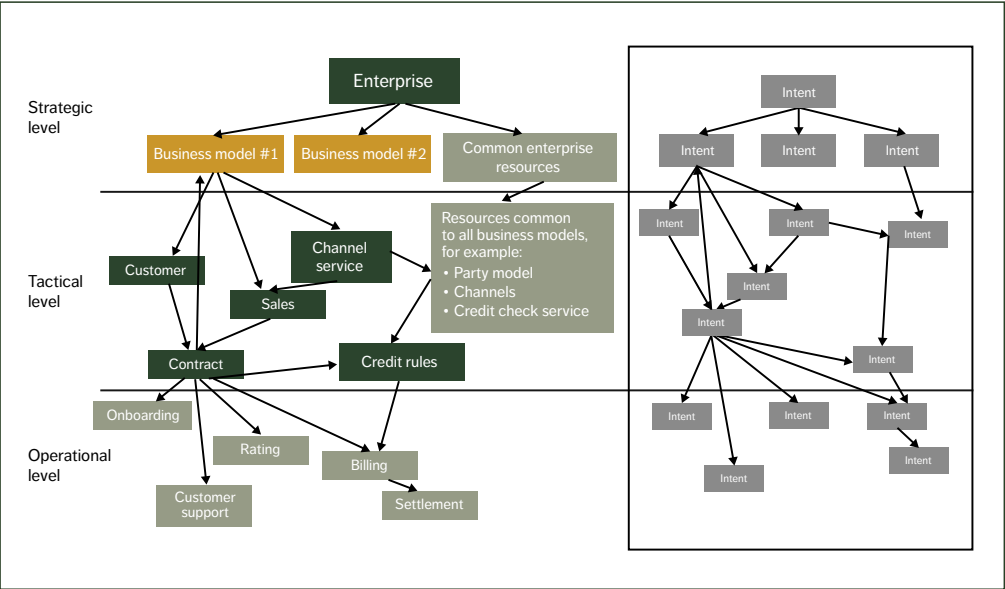


Figure 1 The business intent hierarchy mirrors the business rule hierarchy

Business intents can handle complex and dynamic situations and allow for feedback and comparison of actual and desired outcomes, enabling learning and knowledge building [1].

Higher-level intents in BSS are often expressed as business rules or KPIs. Intents are found at all levels of the business hierarchy, supporting both top- and bottom-line outcomes. An intent can range in complexity from an ‘atomic intent’ to an ‘algorithm of intents’ that combines a set of subintents. The term ‘atomic intent’ refers to the simplest possible intent structure, such as “our company will run a prepaid business model.” Note that an atomic intent on a strategic level

is likely to fan out into several intents on a lower level. While intents can be formulated for both human and machine consumption, they must be stated in a declarative format to facilitate automation in BSS.

Figure 1 illustrates the structure of the business intent hierarchy, which is designed to mirror the business rule hierarchy. An enterprise must have at least three different intent levels – strategic, tactical and operational [5] – all of which are the responsibility of the BSS. In software terms, these levels are equivalent to requirements, design and implementation, and execution.

	Strategic level	Tactical level	Operational level
Recurrence	One-off or few	Enough to learn	Very many
Human interaction	Yes	Yes	No
Learning/reasoning	Reasoning	Learning/reasoning	Execution
Feedback	Limited	Yes, key element	Large volumes
Main constraint	Quality	Quality	Time
Clarity of data use	Undetermined	Limited to own data	Deterministic and limited BSS set
Context, sources and data volumes	Many, external and large volumes	BSS	BSS and inference thereof
Data size	Large	Large but limited	Optimized for the intent
Data types and time series	All types	Many but limited	Few and limited to BSS origin

Figure 2 AI usage patterns are different at strategic, tactical and operational levels

The business strategy level owns and formulates the enterprise’s top intents. For the sake of simplicity, Figure 1 breaks down only one business model to all three levels, but it is important to note that an average-size operator runs several different business models. The business tactic level is responsible for designing and implementing the intents of each of these business models at the operational level. This is normally achieved by breaking down the strategic intent into smaller, digestible parts, such as subintents with associated rules, information and processes. The business operation level – the core of traditional BSS – is responsible for executing to deliver the intents. This level requires further

automation to meet ownership and business transaction cost requirements. All three levels benefit from AI support but have different usage patterns and characteristics, as shown in Figure 2. The differences most worth noting are in terms of repetitions, context, exploration and data characteristics.

The OODA loop

The OODA loop [6] was initially developed in the 1970s as an in-combat decision tool of the U.S. Air Force. OODA stands for observe, orient, decide and act. Many of the OODA loop’s basic concepts are found in today’s software agent systems. The version

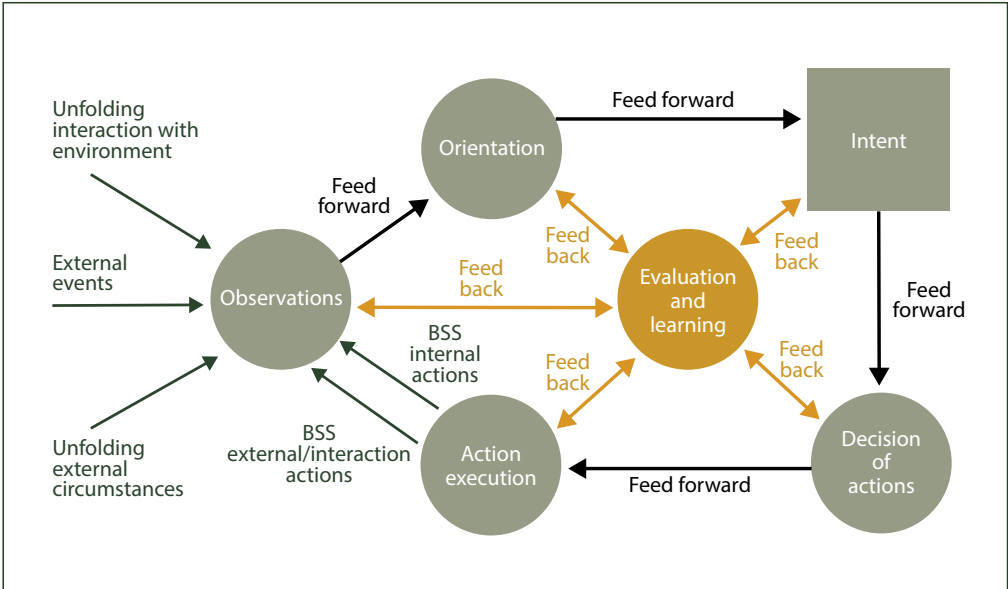


Figure 3 The OODA loop, modified to enable learning and intents

shown in Figure 3 is complemented with explicit intent and learning capability.

The OODA loop idea is quite simple. First, observe or detect changes, events, stimuli or other things that happen in the context of interest, including internal states. Observations can be single or unfolding events, and they can be simple or complex in structure. Depending on the data, AI is often needed to interpret observations. Typical observations in BSS could be the availability of a customer’s usage record or the arrival of a potential customer to a web shop.

The orientation step consists of aggregating and analyzing the observations that form the basis for the

decisions. Analyzing the individual observations and aggregating them into a complete situation description requires multilevel AI support. The orientation step in BSS should enrich the observation with customer data as much as possible. This data enables the BSS to select the correct rating and charging parameters for a particular customer when their usage record becomes available, for example, or to conclude that a visitor to a web shop is looking for a new phone but seems to be price sensitive.

To clearly separate the intent from the decision of action that fulfills the intent, we have added intent to our modified OODA loop. We achieved this by

dividing the traditional OODA decision step into two distinct process steps: intent and decision. Intents in BSS are statements of the desired business outcome in a given situation. In the case of invoicing, this would mean ensuring that the rates/charges on the invoice are in accordance with the customer's contract. In the case of a price-sensitive potential customer visiting a web shop to look for a low-priced phone, the intent would be to convince that individual to buy a phone in the medium-price range rather than choosing the least-expensive option.

The decision is limited by the inventory of available possible actions. A selection is made from the available array of actions that best matches the intent. It is also possible to enrich the decision with simulations to predict the action outcome. The action is simply the execution of the decision. In BSS, actions are carried out by business processes and they result in business outcomes.

Examples of BSS decisions (and resulting actions) would include the decision to apply the standard rate/charge process in the case of a new customer usage record, or the decision to show a price-sensitive web shop visitor not only low-priced phones but also medium-priced models that have received excellent customer ratings.

Evaluation, learning and feedback are essential to build a system with optimal performance that can adapt to both business and enterprise-external changes. An optimal system uses the process of orientation, intents and decisions to continuously compare and evaluate both business outcomes and its own capabilities. Adaption may require new or additional higher-level analysis, algorithm redesign and algorithm retraining. Sometimes, it is enough to have good in-operations learning, such as a continuous algorithm retraining capability. An example of learning in BSS could be reaching the conclusion that when dealing with price-conscious web shop visitors, better outcomes can be achieved by showing a mix of low-price and medium-price phones with good ratings, as opposed to including the expensive phones as well.

The OODA loop can have various depths of reasoning, from deterministic to deep learning – that

●● EVALUATION, LEARNING AND FEEDBACK ARE ESSENTIAL TO BUILD A SYSTEM WITH OPTIMAL PERFORMANCE ●●

is, the same OODA-loop engine can be used to observe, decide and select the proper actions for all event types, regardless of complexity. It can also be used recursively to build layered structures with arbitrary depth – that is, it can support multilayered business processes and interactions.

It is critical that the people working in AI-enabled processes are able to understand the reasoning behind AI-generated results. All of the steps in our modified OODA loop can be understood and executed by both humans and machines – which makes it possible to work together in the most efficient way possible based on the particular circumstances of the organization.

Use case: reducing manual handling of invoices

An invoicing-related use case provides a good illustration of how AI adds value to BSS. In this scenario, a telecom operator notices an increase in the number of invoices that require manual handling, which is costly for the company. The executive team initiates a strategic project to address the issue.

The objective at strategic level is twofold: to establish facts on the invoice situation and to state a future invoice strategy – that is, to set an intent – regarding invoice handling and cost. The strategy team pools information about known challenges in invoicing and gathers external data for benchmarking. With the help of classification and statistical analysis algorithms, the following strategic-level intents for invoices are established by the strategic project:

- » manual handling of less than 1 percent of all invoices
- » average handling cost per invoice of less than USD 1.

This use of two dimensions of intent ensures a sound business balance, helping to avoid potential pitfalls, such as the possibility of reaching 0.0001 percent of manually handled invoices at an average cost of USD 100 per invoice.

Once the strategic intent has been set, work on the tactical level can begin. The primary challenge at this stage is to understand and classify all the reasons why some invoices require manual handling while others don't, and to select the optimal AI algorithms that can deliver results in line with the strategic intents. The tactical level begins by defining an efficient subintent structure and estimating each subintent's yield to the strategic intent, in order to select the most valuable ones. Then, for each subintent, it classifies possible AI algorithms to identify the best ones. Important considerations include:

- » the information requirement and the complexity
- » the volatility of the constituent knowledge components in the problem, which in turn determines whether machine learning is enough or if it must be combined with machine reasoning or deep learning to create deep enough or adaptable algorithms
- » feasibility, effort and automation level in business operations
- » cost estimations, implementation, operation and support.

In this type of invoicing use case, it makes sense to introduce customized communication patterns that vary according to customer character type. Therefore, part of the work at the tactical level involves defining three distinct customer personas – angry, regular and docile complainers, for example – and customize anomaly invoice messages for each of them. The next step is to test these different messages on a small portion (1-3 percent) of the customer population to find the right message for each customer persona to ensure that their complaints are resolved without escalation to manual handling. Selecting only a small fraction of the total customer population reduces the business risk.

Finding the necessary knowledge components is an iterative task that requires AI support and access to relevant information. The tactical level is also responsible for the AI algorithm life cycle including design, implementation and operational launch. Further, it is responsible for stating the necessary changes to BSS, so it can both calculate according to the AI algorithms and automatically execute the new behavior in the invoice functionality.

The tactical-level subintents for the invoicing use case are:

- » invoice input correctness: higher than 99.999 percent
- » invoice anomaly statistics and predictions at both group and individual level: anomaly type, costs, volumes, services and dates
- » customer persona classification into three levels (angry, regular and docile complainers) with less than 1 percent error
- » customized message success rate above 90 percent.

The tactical-level changes to BSS in terms of new rules, information and processes are:

- » calculate invoice anomalies and recheck invoice input if anomaly probability is higher than 15 percent
- » determine customer persona complaint classification with continuous learning capability
- » customize the message success rate to achieve continuous learning capability
- » instruct customer support team to "cut it short" in cases with no anomaly and with angry
- » calculate and expose subintent and intent outcome, along with their projections and variance.

The methodology of the tactical level is similar to that of AI-supported CRISP-DM [7]. Once the tactical level steps have been completed, the role of the operational level is simply to execute the algorithms and the new BSS logic with as much automation as possible. After a short training period, the CSP can sort out the invoices that require

manual handling, identify the root causes, classify customers that systematically complain, and select the most efficient customized message. The error rate and the cost per invoice are initially quite high but decrease rapidly below the strategically stated intent as the invoice communication is tuned.

Implications

The invoicing use case makes it clear that BSS must have an omnipresent AI ability to be able to support strategic and tactical investigations, as well as having the agility in operations to change behavior to accommodate new algorithms, rules, information and processes. The inclusion of business models – that is, the grouping of rules, information and processes tuned to work together to deliver business outcomes for specific business situations – is also critical in the evolution of BSS. The intent structures must mirror the business model structures and their life cycles.

AI-native BSS require an expansion of the business logic elements – rules, information/objects and processes – to include intents and events. The business logic elements, often hidden inside applications, must be externalized to support the conversion of AI findings to automated BSS behavior. The business information must be structured in an ontology and made available to all business support users and applications, AI systems included, into a business information lake.

AI-native BSS must support both run-time and business-design-time. Traditional BSS are primarily

built with few configuration options for run-time, resulting in less agility [5]. While it is true that AI can help a business evolve in running BSS (for example in continuous development and operations mode), there is no avoiding the fact that this requires a BSS architecture that is at least partially new.

Conclusion

It is widely recognized that 5G and the IoT represent the main growth opportunities for communication service providers (CSPs) in the coming decade. To support emerging use cases in these areas, CSPs require business support systems (BSS) that can handle complex business situations and optimize outcomes with minimal manual intervention. Artificial intelligence (AI) is the obvious answer, but introducing it into existing BSS is problematic for a number of reasons. Instead, Ericsson recommends an architectural change to traditional BSS to create AI-native BSS. Most significantly, this evolution requires the inclusion of an enterprise’s strategic, tactical and operational levels in the BSS, together with the introduction of two new business logic elements (intents and events). One of the key differences between traditional BSS and AI-native BSS is the fact that AI-native BSS enable the various applications within the BSS to share business information with each other in an efficient and secure manner – a critical capability in the emerging 5G-IoT world.

Further reading

- » Ericsson, The dawn of machine intelligence, available at: <https://www.ericsson.com/en/news/2017/9/the-dawn-of-machine-intelligence>
- » Ericsson, Zero-touch could herald a new era in service provider customer interaction, available at: <https://www.ericsson.com/en/press-releases/2018/5/ericsson-zero-touch-could-herald-a-new-era-in-service-provider-customer-interaction>

THE AUTHORS

The authors would like to thank Jörg Niemöller for his contributions to this article.

Lars Angelin

◆ is an expert in BSS within Business Area Digital Services at Ericsson. He has more than 30 years of experience in the areas of



concept development, architecture and strategies

within the telco and education industries. Angelin joined Ericsson in 1996 as a research engineer, and in 2003 he moved to a position as concept developer in the M2M and OSS/BSS areas. Since 2006 he has focused on BSS – specifically business support, enterprise architectures and the software architectures to implement BSS systems. He holds an M.Sc. in engineering physics, a Tech. Licentiate in tele-traffic theory from Lund Institute of

Technology in Sweden, and an honorary Ph.D. from Blekinge Institute of Technology in Sweden.



Johan Silvander

◆ is a senior specialist in information management who has worked at Ericsson

for more than 20 years. Over the years, his work has focused on the areas of OSS and BSS in a variety of different roles, including serving as a member of core architecture teams, working as a designer, taking technical responsibility for integration and installation projects, and being a test leader. He holds a Tech. Licentiate in computer science from Blekinge Institute of Technology, where he is currently pursuing a Ph.D.

References

1. TM Forum, Open Digital Architecture, 2018, available at: <https://www.tmforum.org/resources/whitepapers/open-digital-architecture/>
2. Ericsson, Unlocking 5G’s revenue potential: a roadmap for operators (press release), February 26, 2018, available at: <https://www.ericsson.com/en/press-releases/2018/2/unlocking-5gs-revenue-potential-a-roadmap-for-operators>
3. Forbes, How Artificial Intelligence Is Revolutionizing Business In 2017, September 10, 2017, Louis Columbus, available at: <https://www.forbes.com/sites/louiscolombus/2017/09/10/how-artificial-intelligence-is-revolutionizing-business-in-2017/#58ebeab25463>
4. Harvard Business Review, Artificial Intelligence for the Real World, January-February 2018, Thomas H. Davenport and Rajeev Ronanki, available at: <https://hbr.org/2018/01/artificial-intelligence-for-the-real-world>
5. McKinsey, Smarter analytics for banks, September 2018, Carlos Fernandez Naviera et al., available at: <https://www.mckinsey.com/industries/financial-services/our-insights/smarter-analytics-for-banks?cid=other-eml-alt-mjp-mck-oth-1810&hlkid=d3be9327efb84ecb44a1d8d391d0d8f&hctky=2669978&hdpid=ddc7fd1c-3815-4675-ae1a-f53e67d88452>
6. OODA loop definition available at: https://en.wikipedia.org/wiki/ODA_loop
7. CRISP-DM definition available at: https://en.wikipedia.org/wiki/Cross-industry_standard_process_for_data_mining

OPEN, INTELLIGENT AND

model-driven: evolving OSS

The simplicity required to deploy and manage services in future network and infrastructure operations is driving the need to rethink traditional operations support systems (OSS). To unleash the potential of 5G, Network Functions Virtualization (NFV) and software-defined networking (SDN), future OSS need to be open, intelligent and able to support model-driven automation.

MALGORZATA SVENSSON,
MUNISH AGARWAL,
STEPHEN TERRILL,
JOHAN WALLIN

The long-standing paradigm of bundling vendor and domain-specific management with network functions has been challenged in recent years by a new approach that is built on the concept of an open and model-driven platform. This new approach leverages open source and uses standardized interfaces to enable a horizontal management platform.

■ Today's service providers expect rapid network deployments, agile introduction of new services and cost-efficient operation and management – requirements that are even more important when planning for future 5G capabilities. Recent technology trends make it possible to redefine traditional support systems at the same time as

they enable greater efficiency in development and operations.

For example, model-driven automation eliminates the need for manual interaction by externalizing behavioral aspects of specific domains from application logic. Real-time and near-real-time analytics redefine the implementation and scope of support system functions like performance, fault management, assurance and optimization, where analytics models act on the collected and correlated data, producing insights that are far richer in scope than the outcome of the traditional functions. Machine intelligence (MI) algorithms enable the transition from reactive to proactive decision making.

The DevOps paradigm simplifies development and operational processes. It requires development and deployment-friendly software architecture and

tool chains that support automation. Microservice architecture supports the DevOps paradigm and enables highly scalable, extendable and flexible systems.

Ericsson's approach to evolving OSS is based on combining these technologies and solutions to create a modern OSS architecture that is optimized to meet the requirements of service providers in the mid to long term.

Our OSS architecture principles

The key principles guiding Ericsson's OSS architecture concept are that it is service oriented, that the automation is analytics and policy driven, and that it uses virtualization and abstraction of network functions. By following these principles, we can evolve OSS to become programmable and able to leverage the interfaces offered by the production domains.

Definitions of key concepts

- » **Analytics** is the discovery, interpretation and communication of meaningful patterns in data. It is the umbrella for AI, ML and MI.
- » **Cloud native** is a term that describes the patterns of organizations, architectures and technologies that consistently, reliably and at scale take full advantage of the possibilities of the cloud to support cloud-oriented business models.
- » **Microservice** is a small service supporting communication over network interfaces. Several such services constitute an application. Each microservice covers a limited and coherent functional scope and is independently manageable by fully automated life-cycle machinery. A microservice is small enough to be the responsibility of a single, small development team.
- » **OSS business layer** is a conceptual layer of the OSS that guides the service provider through the operational support subset of its business process.
- » **OSS control layer** is a layer of the OSS containing functionality that controls and manages production domains.
- » **Policy** is a function that governs the choices in the behavior of a system. Policy can use a declarative or imperative approach.
- » **Programmability** is the ability to externally influence the behavior of a system in a defined manner.
- » **Resource** is a means to realize a service, and can be either physical or non-physical.
- » **Service** is a means to deliver value to a customer or user; it should be understandable to a customer. Service is also a representation of the implementation aspects to deliver the value.

Terms and abbreviations

AI – artificial intelligence | **API** – application programming interface | **BSS** – business support systems | **ETSI** – European Telecommunications Standards Institute | **MANO** – Management and Orchestration | **MI** – machine intelligence | **ML** – machine learning | **NFV** – Network Functions Virtualization | **ONAP** – Open Network Automation Platform | **OSS** – operations support systems | **PNF** – Physical Network Function | **SDN** – software-defined networking | **SLA** – Service Level Agreement | **TOSCA** – Topology and Orchestration Specification for Cloud Applications | **UDM** – Unified Data Management | **VNF** – Virtual Network Function

●● AUTOMATION CAN BE FURTHER ENHANCED BY APPLYING MI, WHICH IS THE COMBINATION OF MACHINE LEARNING AND ARTIFICIAL INTELLIGENCE ●●

Abstraction, programmability and domain-specific extensions

Networks are complex. As the physical resources within them transform into virtual resources such as Virtual Network Functions (VNFs), virtual switches, virtual networks and virtual cloud resources, they will expose only what is necessary and be managed in a uniform way. This leads to a logical representation of resources and networks, which is referred to as abstraction. The capabilities of these virtual resources are exposed through their interfaces, while their behavior is influenced by these interfaces. This is referred to as programmability. The methodology of abstraction and programmability is continually and recursively applied at all levels of the OSS and networks, from the resources' interfaces to the exposed services.

Abstraction and programmability are the main prerequisites for uniform management of automation, easing the shift from managing networks to managing automation. They enable simplification by exposing only the required capabilities of a domain, while still supporting an efficient interaction between management system scopes, and management systems and networks.

Regardless of the generalization of the behavior between various production domains, the domain-specific aspects remain. Some examples of domain-specific extensions are: managing and optimizing radio frequency, designing and handling data center capacity, or designing L2/L3 service overlay. Specific production domain competencies and processes are required to provide the necessary capabilities in these examples, even though the capabilities are realized by the same OSS.

From automatic management to management of automation

The goal of automation is to provide a zero-touch network, which means that automation moves from automating via scripting what can be done manually, to an autonomic network that takes care of itself and handles previously unforeseen situations. This changes the approach to management, from managing networks manually to managing automation.

A key part of managing automation is the control loop paradigm, as shown in *Figure 1*. This is achieved by designing the insights and policies required for a use case, and the decisions that need to be made. Insights are the outcome when data is processed by analytics. Policies represent the rules governing the decisions to be made by the control loop. The decisions are actuated by COM (Control, Orchestration, Management), which interacts with production domains by requesting actions such as update, configure or heal. The control loops are implemented by multiple OSS functions and can act in a hierarchical way.

By leveraging analytics, MI and policy, control loops become adaptive and are central to assuring and optimizing the deployed services and resources.

MI and autonomic networks

Automation can be further enhanced by applying MI, which is the combination of machine learning (ML) and artificial intelligence (AI). MI adapts to the situation in a network and learns to provide the best insights for a given network situation.

As the level of MI increases, the role of policy shifts from governance of the decisions to be made, to ensuring that the insights produced from MI are appropriate. In this sense, MI is an enabler for autonomic networks.

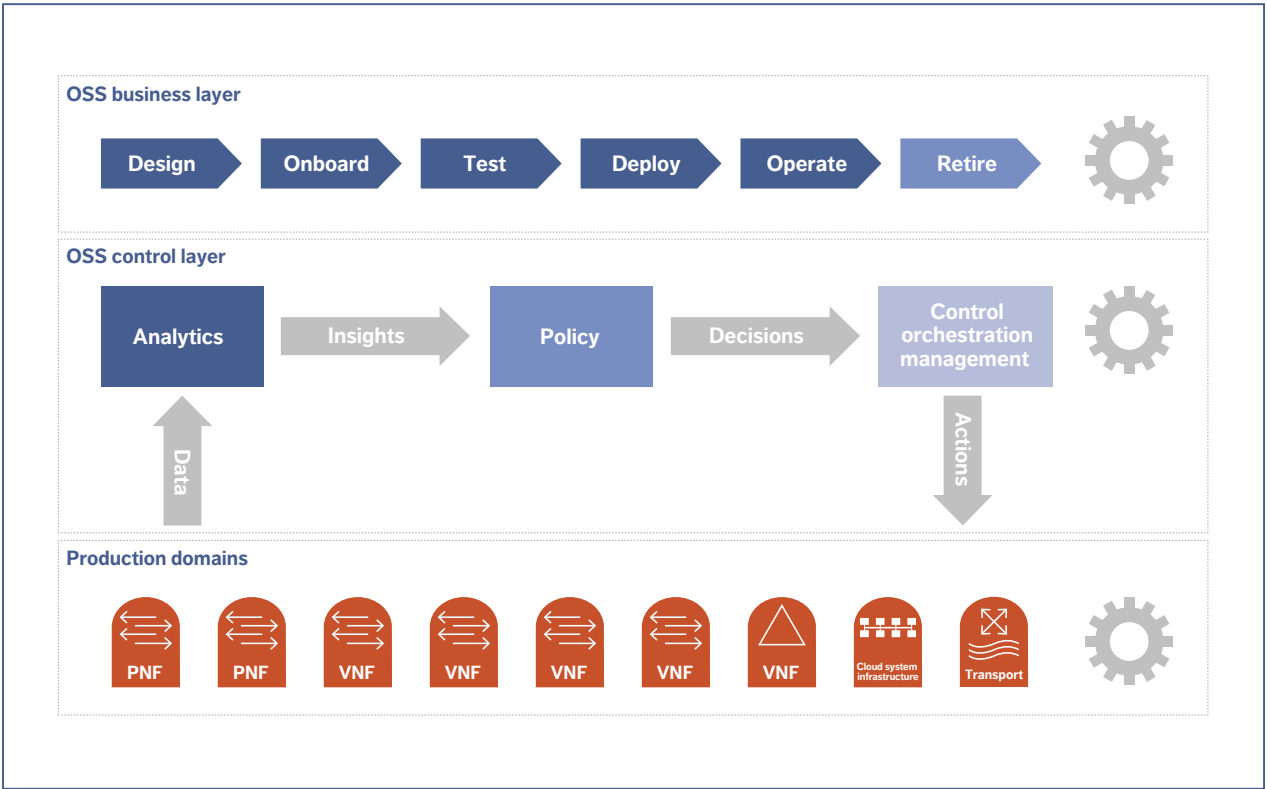


Figure 1 Analytics and policy-driven automation: closed control loop

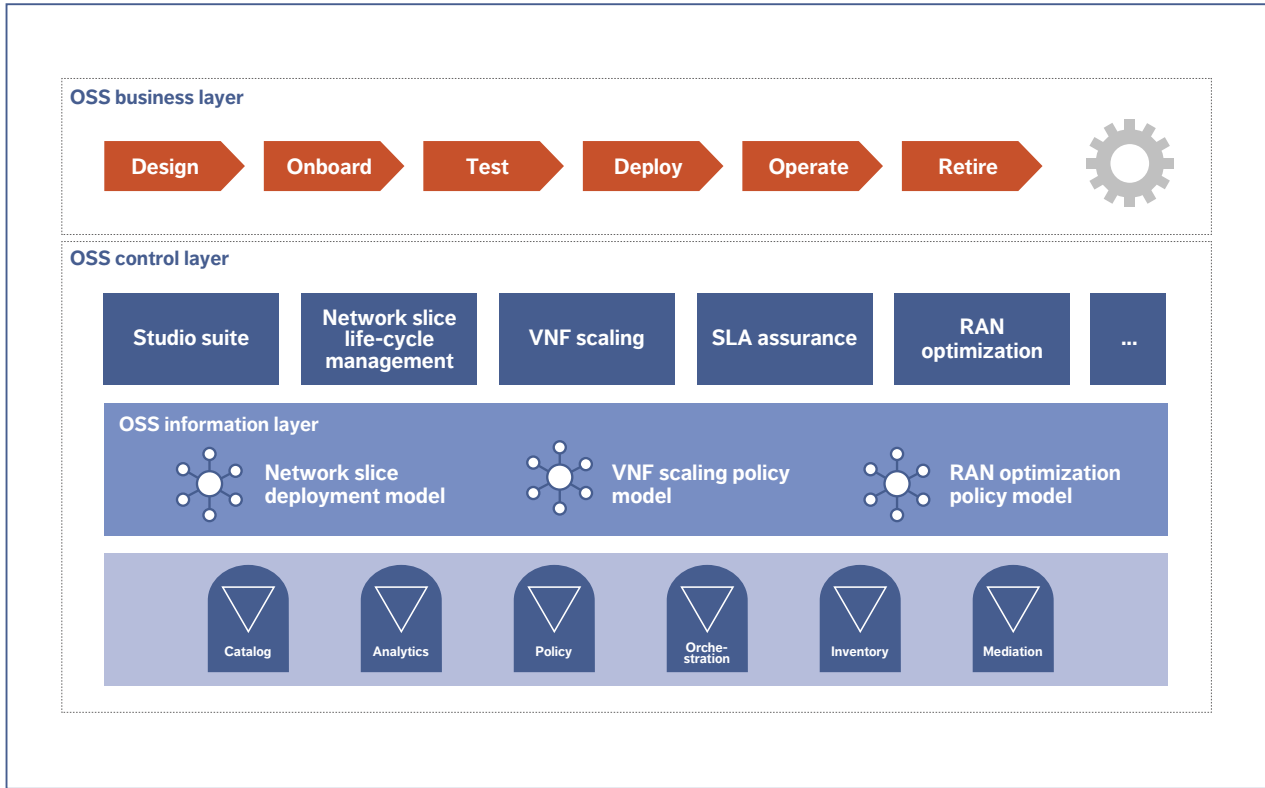


Figure 2 Model-driven OSS architecture

Model-driven architecture

To secure efficient and consistent management, the behavioral aspects of specific domains are externalized from application logic to models. There are models for configuration, orchestration, policy rules and analytics.

Resource and service life-cycle management is a good example of a model-driven approach. As shown in [Figure 2](#), the models describe the services and the resources. These models are designed and onboarded in a studio suite. They are then used to test, deploy, operate and retire the services and the resources. The models are based on standard modeling approaches. For example, deployment models are described using HEAT

(specifically, orchestration in OpenStack [1]) – and TOSCA [2]. The “operate” process can cover activities such as scale, upgrade, heal, relocate, stop and start.

The configuration and instrumentation of the executing resource and service instances require resource and service views expressed also as models. IETF YANG [3] has emerged as the mainstream modeling language for this purpose.

Policy rule sets define the behavioral aspects of service assurance, resource performance and scaling. Since the models are the way to capture business logic and intellectual property, they will become sellable objects themselves and be subject to commercial agreements.

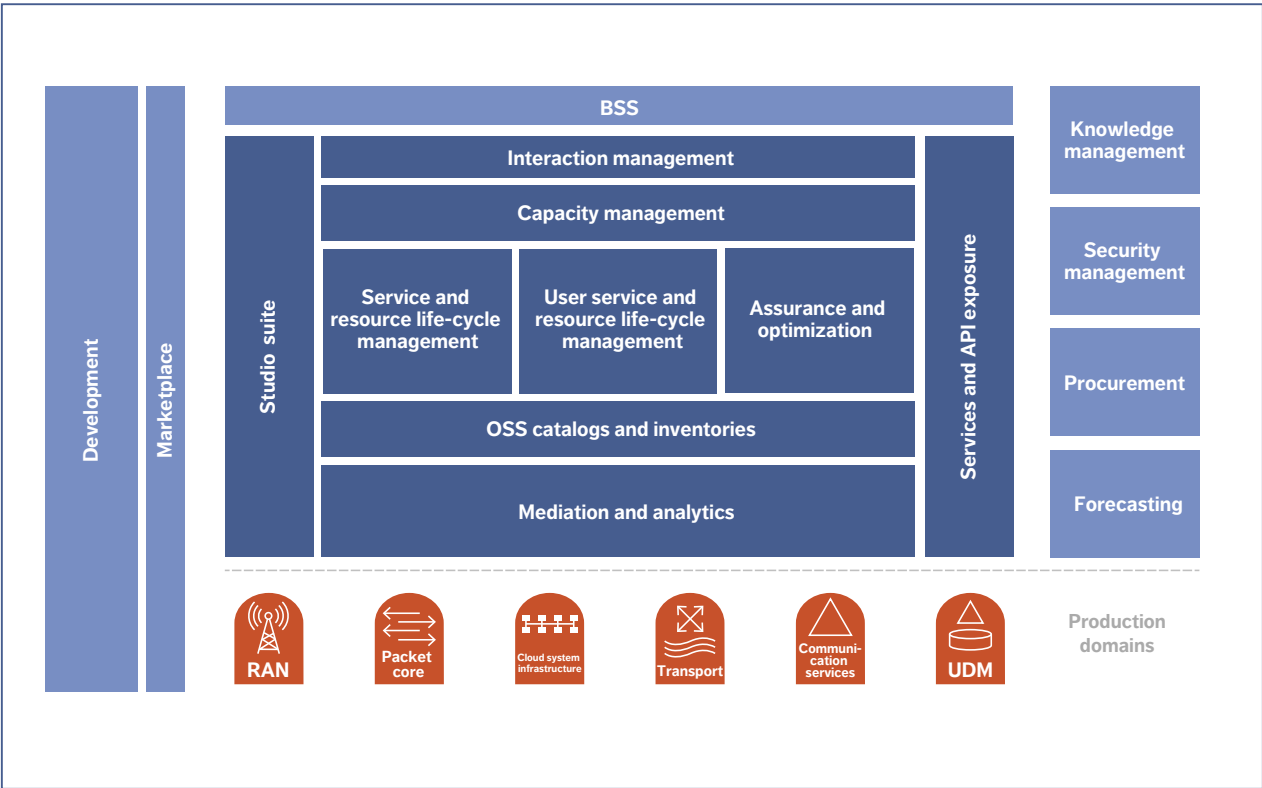


Figure 3 Ericsson's OSS architecture

Modular architecture and implementation capabilities

The optimal OSS of the future will be based both on the principles outlined above and on modular architecture and implementation capabilities.

Studio suite

[Figure 3](#) illustrates the studio suite, which provides a single, coherent environment to design, test, operate, control and coordinate all the business, engineering and operational processes of a service provider. This is done by expressing the processes, interface extensions, policies, deployments and configurations as models and using them for automation of all the OSS functions and the

managed production domains. The studio suite framework includes business, engineering and operational studios.

The business studio is the environment where business and operational processes are maintained. The business studio automatically orchestrates the process models by invoking operations on the appropriate functions in the OSS stack.

The engineering studio provides a model-driven platform where the OSS functions and the relevant information and information models associated with those functions are constructed and extended. It lets a user (human or machine) realize various management tasks by extending the OSS functions with use-case specific models for configuration,

orchestration, data and policies. It provides software development kits for defining the extension models and then instantiates the needed components, provisions the components with the models and wires them together to realize the desired use case. An example could be a set of ML functions that provide scalable machinery that can be assigned to multiple uses by applying different configuration and data models.

The operational studio offers capabilities to manage resources and services throughout their life cycle. Resources represent the infrastructure and the networks of production domains. Services use the infrastructure and networks to deliver value to customers. One of the first activities in the resource and service life cycle is to construct a specification of a resource or a service, then assign the required policy rules and configurations defining the behavior of the network, or the infrastructure or the service. Once they are defined, the specifications, policies and configurations are stored in the OSS catalogs and distributed in real time to be used by the OSS functions.

Managing services and resources

Service and resource life-cycle management functions provide a programmable way of managing the services and resources of the production domains based on specifications, policies and configurations stored in the catalogs. The core capabilities are orchestration and policy engines that parse the defined models to realize all the life-cycle steps of the services and resources. The policy and orchestration engines automate the network management by taking over the burden of low-level decision making and execution of life-cycle operations from human users.

These functions are triggered by events generated

by surrounding functions like interaction management, assurance and optimization. They provide both closed loop optimization capabilities (where decision making is done based on policies and MI) and an open loop optimization feature (where a human user decides based on a set of automatic recommendations). This dynamic, model-driven and real-time interaction is what will make this future OSS solution different from the traditional one.

We can already see the need for seamless interaction between OSS software development and operations, which suggests that the core DevOps capabilities of continuous integration, delivery, release and deployment on-demand will be an integral part of the future OSS.

OSS inventories will be used to store instantiated resources and services. They track the presence, capacity and configuration of resources and services, as well as their relationships to other resources and services. They provide the data for capacity management, which enables capacity planning, design and monitoring over time. By using analytics in capacity management, it is possible to make it work in real time and be predictive and self-learning.

The purpose of the user service and resource life-cycle management function is to manage resources and services that have been assigned to a user, tenant or subscriber. This capability also supports user, tenant and subscriber provisioning. The closed and open control loop architecture patterns apply here as well. The user service and resource life-cycle management function has the same properties as the service and resource life-cycle management function.

Mediation provides an open and adaptable interface to collect data, process events generated by the production domains, and do the required translation, transformation and filtering. It also provides a set of flexible interfaces to invoke life-cycle

operations on the networks and infrastructures of the production domains.

The assurance function makes it possible to check that resources and services behave as they have been designed, published and offered. Assurance makes use of analytics to discover insights about the performance of services and resources. Policy engines are then triggered to act on the insights to determine actions that ensure the desired performance level offered on services and resources.

Our OSS architecture is designed to use interaction management, services and application programming interface (API) exposure to interact with external functions. A few examples of those external functions are development, marketplaces, business support systems, advanced security and knowledge management systems, procurement and forecasting.

The production domain example shown in Figure 3 includes a RAN, packet core and communication services, Unified Data Management, cloud system infrastructure, fixed access and transport networks. Our OSS manages multi-vendor network and infrastructure.

The OSS functions are based on an API-first approach that mandates that an API be defined before the function-exposing services can be implemented. The approach enables parallel development and provides the ability to adapt to external implementations. The API management framework facilitates the effective use of APIs during development, discovery of internal API endpoints at runtime and efficiently controlled exposure of APIs to external applications.

Cloud native

Cloud native is the new computing paradigm that is optimized for modern, distributed systems

environments, capable of scaling to tens of thousands of self-healing, multi-tenant nodes. Cloud native systems are container packaged, dynamically managed and microservice oriented. The Ericsson OSS of the future will be based on cloud-native architecture to enable better reuse, simplified operations and improved efficiency, agility and maintainability.

The microservice architecture enables fine-grained reuse by having several small components instead of a single large one. By being loosely coupled and backward compatible, the services can be developed independently, enabling efficient DevOps. It also allows each service to choose the most efficient development and runtime technology for its purpose. The microservice architecture makes it possible to compose various business solutions, then deploy them automatically, significantly reducing deployment cost and time to market. The architecture will use the cloud-native ecosystem as a portability layer to support multiple deployment options.

Security

Supporting multiple deployment models requires multiple security capabilities like credential management, secure communication, encryption, authentication and authorization, which can be selected based on the deployment. Our OSS architecture will include the capabilities needed to support the multiple deployment models and to protect the models.

Embracing industry initiatives

Due to NFV and programmable networks such as SDN, there is huge industry interest in automation to decrease complexity and achieve rapid introduction of services and resources. This has resulted in several significant industry initiatives, most notably

ETSI-MANO [4] and the Open Network Automation Platform (ONAP) [5].

Initiatives that call for standardization provide an environment to develop strong concepts and identify important interfaces, while open source provides reference implementations, as well as implementation interfaces.

ONAP

By bringing together the resources of its members, ONAP has sped up the development of a globally shared architecture and implementation for network automation. ONAP provides both design time and runtime reference implementations, covering service design and creation, catalog, inventory, orchestration and control, policies and analytics. It also includes service management and automation capabilities for Physical/Virtual Network Functions (PNFs/VNFs), transport and cloud infrastructure.

Ericsson’s modular and model-driven OSS approach is well aligned with ONAP and extends the ONAP vision to automation beyond network management to include interaction management, workforce management, user service and resource life-cycle management, advanced service optimization and assurance as a few examples. To interact with ONAP, we are in the process of adopting the necessary interfaces and model definitions in our product portfolio (both in OSS and network functions). Ericsson is committed to ONAP and heavily engaged with it across a broad range of topics. We aim to use the ONAP-provided open source implementations in accordance with Ericsson’s architecture principles.

Managing multiple production domains requires a model-driven architecture because achieving domain-specific behavior requires the application of

appropriate models on a compatible OSS platform. Since a model-driven architecture is essential to drive automation, we are working toward ensuring that our models are usable within ONAP.

Conclusion

As both an established infrastructure and OSS supplier, Ericsson has a vision for OSS in which autonomic networks enable intelligent and automated service and resource life-cycle management. We are using analytics, ML, policy and orchestration technologies to create the OSS of the future, and to offer greater efficiency in development and operations. We apply a DevOps paradigm to our software development to keep us close to customers and speed up our software delivery.

As our approach is model-driven, OSS behavior is externalized from the management platform and materialized through a combination of models and configurations. As a result, the developed models are independent of the execution platform.

Our OSS concept is modular by design and follows microservice architecture principles that make it possible to replace software components with open source implementations. The concept is built on a solid implementation architecture that enables the use of industry-defined interfaces and open source modules, as well as integration with full component compatibility.

Ericsson has embraced open source implementations through active contributions and usage, as well as driving important standardizations. We are committed to driving industry alignment that will help create a healthy ecosystem. 🌐

References

1. **OpenStack, Heat Orchestration Template (HOT) Guide**, available at: https://docs.openstack.org/heat/ocata/template_guide/hot_guide.html
2. **OASIS, OASIS Topology and Orchestration Specification for Cloud Applications (TOSCA) TC**, available at: https://www.oasis-open.org/committees/tc_home.php?wg_abbrev=tosca
3. **Internet Engineering Task Force (IETF), October 2010, YANG – A Data Modeling Language for the Network Configuration Protocol (NETCONF)**, available at: <https://tools.ietf.org/html/rfc6020>
4. **ETSI, Network Functions Virtualization**, available at: <http://www.etsi.org/technologies-clusters/technologies/nfv>
5. **ONAP**, available at: <https://www.onap.org/>

Further reading

» **Ericsson Technology Review, Technology trends driving innovation – five to watch, 2017**, Ekudden, E: <https://www.ericsson.com/en/publications/ericsson-technology-review/archive/2017/technology-trends-2017>

» **Ericsson Technology Review, Generating actionable insights from customer experience awareness, September 2016**, Niemöller, J; Sarmonikas, G; Washington, N: <https://www.ericsson.com/en/publications/ericsson-technology-review/archive/2016/generating-actionable-insights-from-customer-experience-awareness>

» **Ericsson Technology Review, DevOps: fueling the evolution toward 5G networks, April 2017**, Degirmenci, F; Dinsing, T; John, W; Mecklin, T; Meirosu, C; Opsenica, M: <https://www.ericsson.com/en/publications/ericsson-technology-review/archive/2017/devops-fueling-the-evolution-toward-5g-networks>

» **Ericsson, Gearing up support systems for software-defined and virtualized networks, June 2015**: <https://www.ericsson.com/en/news/2015/6/gearing-up-support-systems-for-software-defined-and-virtualized-networks->

» **Ericsson, Zero touch networks with cloud-optimized network applications**, Darula, M; Más, I: <https://www.ericsson.com/assets/local/narratives/networks/documents/zero-touch-networks-with-the-5g-cloud-optimized-network-applications.pdf>

» **Ericsson Technology Review, Architecture evolution for automation and network programmability, November 2014**, Angelin, L; Basilier, H; Cagenius, T; Más, I; Rune, G; Varga, B; Westerberg, E: <https://www.ericsson.com/en/publications/ericsson-technology-review/archive/2014/architecture-evolution-for-automation-and-network-programmability>

THE AUTHORS

Malgorzata Svensson

◆ is an expert in OSS. She joined Ericsson in 1996 and has worked in various areas within research and



development. For the past 10 years, her work has focused on architecture evolution. She has broad experience in business process, function and information modelling; information and cloud technologies; analytics; DevOps processes; and tool chains. She holds an M.Sc. in technology from the Silesian University of Technology in Gliwice, Poland.

Munish Agarwal

◆ is a senior specialist in implementation architecture



for OSS/BSS. He has 19 years of experience in telecommunications working with product development, common platforms and architecture evolution. He has hands-on experience with mediation, rating, order management and interactive voice response. Agarwal holds a Bachelor of Technology degree from the Indian Institute of Technology Kharagpur.

Stephen Terrill

◆ has more than 20 years of experience working with telecommunications architecture, implementation and industry engagement. His work has included both architecture definition and



posts within standardization organizations such as ETSI, 3GPP, ITU-T (ITU Telecommunication Standardization Sector) and IETF (Internet Engineering Task Force). In recent years, his work has focused on the automation and evolution of OSS, and he has been

engaged in open source on ONAP's Technical Steering Committee. Terrill holds an M.Sc., a B.E. (Hons.) and a B.Sc. from the University of Melbourne, Australia

Johan Wallin

◆ is an expert in network management. He joined Ericsson in 1989 and has worked in several areas in R&D in various positions covering design, product management and system



management. He has broad experience of systems architecture in various mobile telephony systems including GSM, TDMA and CDMA. For the past 10 years, he has been working with network management architectures from an overall Ericsson perspective. He holds an M.Sc. in computer engineering and computer science from the Institute of Technology at Linköping University, Sweden.

The authors would like to acknowledge the contribution their colleagues John Quilty, Bo Åström, Ignacio Más, Jan Friman, and Jaco Fourie made to the writing of this article.



Privacy-aware machine learning

WITH LOW NETWORK FOOTPRINT

Federated learning makes it possible to train machine learning models without transferring potentially sensitive user data from devices or local deployments to a central server. As such, it addresses privacy concerns at the same time that it improves functionality. Depending on the complexity of the neural network, it can also dramatically reduce the amount of data needed while training a model.

KONSTANTINOS VANDIKAS, SELIM ICKIN, GAURAV DIXIT, MICHAEL BUISMAN, JONAS ÅKESON

Reliance on artificial intelligence (AI) and automation solutions is growing rapidly in the telecom industry as network complexity continues to expand. The machine learning (ML) models that many mobile network operators (MNOs) use to predict and solve issues before they affect user QoE are just one example.

■ An important aspect of the 5G evolution is the transformation of engineered networks into continuous learning networks in which self-adapting, scalable and intelligent agents can work independently to continuously improve quality and performance. These emerging “zero-touch networks” are, and will continue to be, heavily dependent on ML models.

The real-world performance of any ML model depends on the relevance of the data used to train it.

Conventional ML models depend on the mass transfer of data from the devices or deployment sites to a central server to create a large, centralized dataset. Even in cases where the computation is decentralized, the training of conventional ML models still requires large, centralized datasets and misses out on using computational resources that may be available closer to where data is generated.

While conventional ML delivers a high level of accuracy, it can be problematic from a data security perspective, due to legal restrictions and/or privacy concerns. Further, the transfer of so much data requires significant network resources, which means that lack of bandwidth and data transfer costs can be an issue in some situations. Even in cases where all the required data is available, reliance on a centralized dataset for maintenance and retraining purposes can be costly and time consuming.

For both privacy and efficiency reasons, Ericsson

believes that the zero-touch networks of the future must be able to learn without needing to transfer voluminous amounts of data, perform centralized computation and/or risk exposing sensitive information. Federated learning (FL), with its ability to do ML in a decentralized manner, is a promising approach.

To better understand the potential of FL in a telecom environment, we have tested it in a number of use cases, migrating the models from conventional, centralized ML to FL, using the accuracy of the original model as a baseline. Our research indicates that the usage of a simple neural network yields a significant reduction in network utilization, due to the sharp drop in the amount of data that needs to be shared.

As part of our work, we have also identified the properties necessary to create an FL framework that can achieve the high scalability and fault tolerance required to sustain several FL tasks in parallel. Another important aspect of our work in this area has been figuring out how to transfer an ML model

that addresses a specific and common problem, pretrained within an FL mechanism on existing network nodes to newly joined network nodes, so that they too can benefit from what has been learned previously.

The concept of federated learning

The core concept behind FL is to train a centralized model on decentralized data that never leaves the local data center that generated it. Rather than transferring “the data to the computation,” FL transfers “the computation to the data.”[1]

In its simplest form, an FL framework makes use of neural networks, trained locally as close as possible to where the data is generated/collected. Such initial models are distributed to several data sources and trained in parallel. Once trained, the weights of all neurons of the neural network are transported to a central data center, where federated averaging takes place and a new model is produced and communicated back to all the remote neural networks that contributed to its creation.

Terms and abbreviations

AI – Artificial Intelligence | AUC – Area Under the Curve | FL – Federated Learning | ML – Machine Learning | MNO – Mobile Network Operator | ROC – Receiver Operating Characteristic

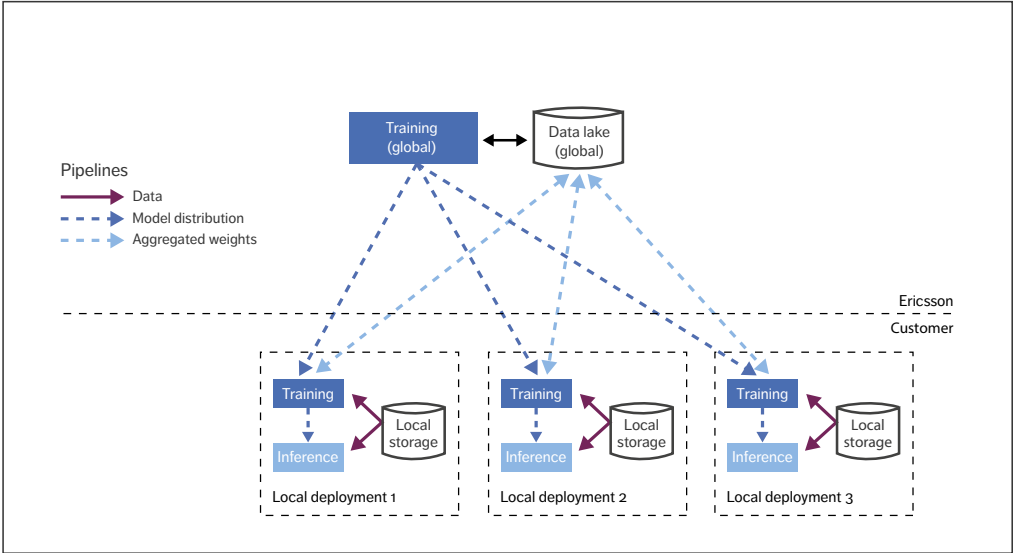


Figure 1 Overview of federated learning

Techniques such as secure aggregation [2] and differential privacy [3] can be applied to further ensure the privacy and anonymity of the data origin.

FL can be used to test and train not only on smartphones and tablets, but on all types of devices. This makes it possible for self-driving cars to train on aggregated real-world driver behavior, for example, and hospitals to improve diagnostics without breaching the privacy of their patients.

Figure 1 illustrates the basic architecture of an FL life cycle. The light blue dashed lines indicate that only the aggregated weights are sent to the global data lake, as opposed to the local data itself, as is the case in conventional ML models. As a result, FL makes it possible to achieve better utilization of resources, minimize data transfer and preserve the privacy of those whose information is being exchanged.

The main challenge with an FL approach is that the transition from training a conventional ML

model using a centralized dataset to several smaller federated ones may introduce a bias that impacts the accuracy originally achieved by using a centralized dataset. The risk for this is greatest in less reliable and more ephemeral federations that span over to mobile devices.

It is reasonable to expect data centers used by MNOs to be significantly more reliable than devices in terms of data storage, computational resources and general availability. However, it is important to ensure high fault tolerance, as corresponding processes may still fail due to lack of resources, software bugs or other issues.

Federated learning framework design

Our FL framework design concept is cloud-native, built on a federation of Kubernetes-based data centers located in different parts of the world. We assume restricted access to allow for the execution of certain processes that are vital to FL.

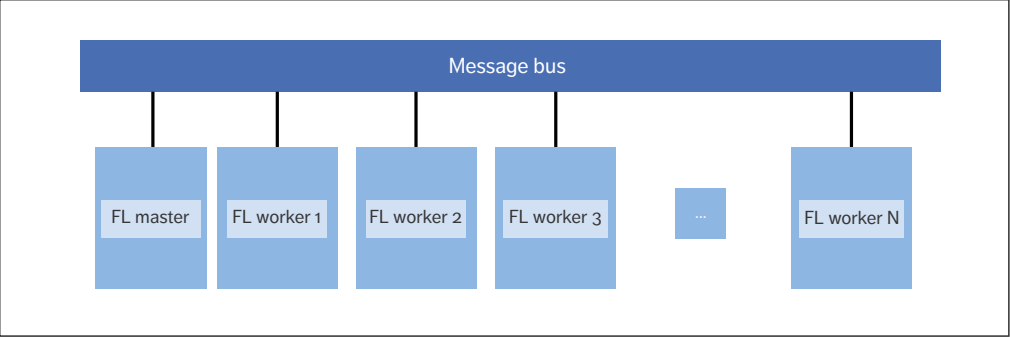


Figure 2 Basic design of an FL platform

Figure 2 illustrates the basic system design. A federation is treated as a task run-to-completion, enabling a single resource definition of all parameters of the federation that is later deployed to different cloud-native environments. The resource definition for the task deals both with variant and invariant parts of the federation.

The variant parts handle the characteristics of the FL model and its hyperparameters. The invariant parts handle the specifics of common components that can be reused by different FL tasks. Invariant parts include a message queue, the master of the FL task and the workers to be deployed and federated in different data centers.

Workers (processes running in local deployments) are tightly coupled with the underlying ML platform that is used to train the model, which is immutable during the federation. In our FL experiments, we selected TensorFlow to train the neural network, which is designed to be interchangeable with other ML platforms such as PyTorch. Communication between the master and the workers is protected using Transport Layer Security encryption with one-time generated public/private keys that are discarded as soon as an FL task is completed.

Invariant components can be reused by different FL tasks. FL tasks can run sequentially or in parallel depending on the availability of resources. Master

and worker processes are implemented as stateless components. This design choice leads to a more robust framework, since it allows for an FL task to fail without affecting other ongoing FL tasks.

Fault tolerance

To reduce the complexity of the codebase for both the master of the FL task and the workers and to keep our implementation stateless, we chose a message bus to be the single point of failure in the design of our FL framework. This design choice is further motivated by the research into creating highly scalable and fault-tolerant message buses by combining leader-election techniques and/or by relying on persistent storage to maintain the state of the message queue in case of a failure [4].

The message exchange between the master of the FL task and the workers is implemented in the form of assigned tasks such as “compute new weights” and “average weights.” Each task is pushed into the message queue and has a direct recipient. The recipient must acknowledge that it has received the task. If the acknowledgement is not made, the task remains in the queue. In case of a failure, messages remain in the message queue while Kubernetes restarts the failed process. Once the process reaches a running state again, the message queue retransmits any unacknowledged tasks.

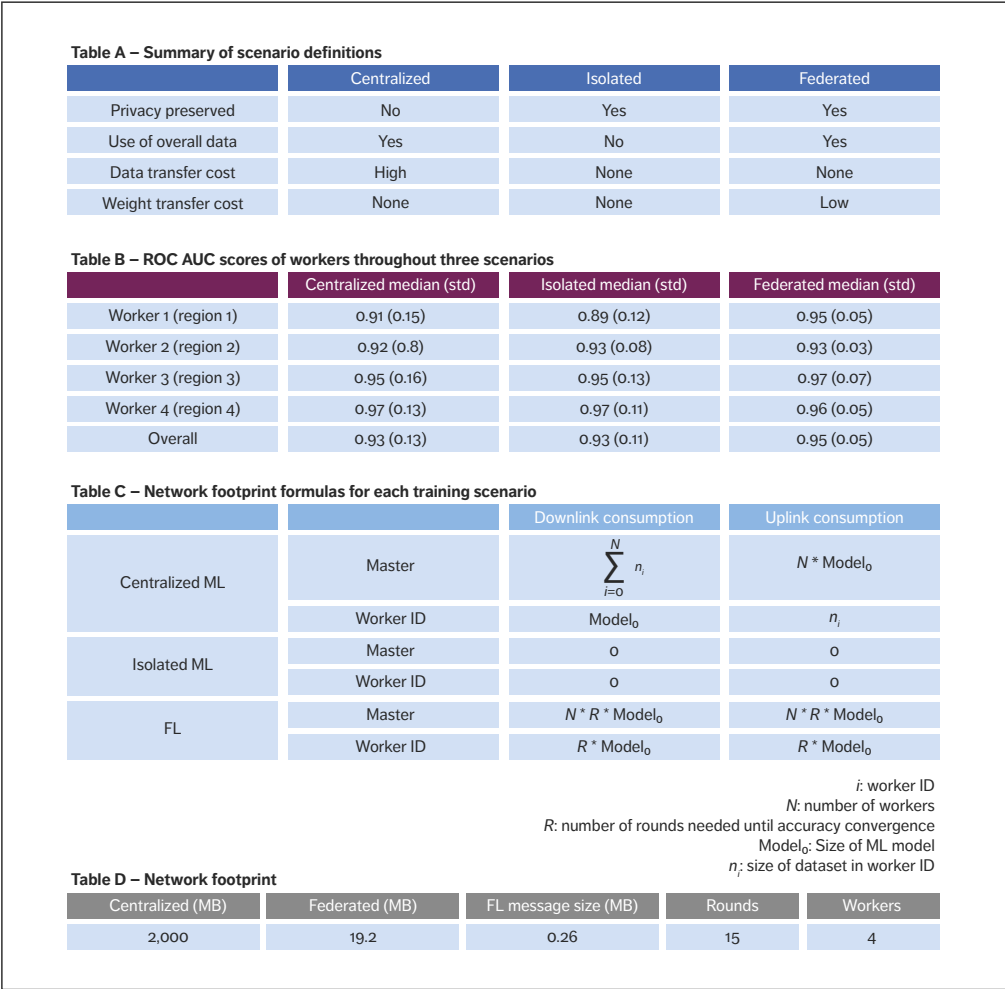


Figure 3 Tables relating to the hardware fault prediction use case

Preventive maintenance use case

Hardware fault prediction is a typical ML use case for an MNO. In this case, the aim is to predict whether there will be a hardware fault at a radio unit within the next seven days based on data generated in the eight-week interval preceding the prediction time. The inputs to the ML model consist of more than 500 features that are aggregations of multiple performance management counters, fault management data such as alarms, weather data and the date/time since the hardware has been active in the field.

Three training scenarios

We performed the experiments in three scenarios – centralized ML, isolated ML and FL.

Centralized ML is the benchmark scenario. The datasets from all four worker nodes are transferred to one master node, and model training is performed there. The trained model is then transferred and deployed back to the four worker nodes for inference. In this scenario, all worker nodes use exactly the same pretrained ML model.

In the isolated ML scenario, no data is transferred from the worker nodes to a master node. Instead, each worker node trains on its own dataset and operates independently from the others.

In the FL scenario, the worker nodes train on their individual datasets and share the learned weights from the neural network model via the message queue. The saturation of the model accuracies is achieved after 15 rounds of the weight-sharing and weight-averaging procedure. In this way, the worker nodes can learn from each other without transferring their datasets.

The properties of each training scenario are summarized in [Figure 3](#), Table A.

Accuracy results

Table B in [Figure 3](#) presents the results in the form of median ROC AUC (receiver operating characteristic area under the curve) scores obtained through more than 100 independent experiments. The scores achieved in the FL scenario are similar to those achieved in the centralized and isolated ones, while the variance of the FL scores is significantly lower compared with the other two scenarios.

The results in Table B show that it is worker 1 (south) that benefits from FL. They also suggest that an isolated ML approach can be recommended in cases where the individual datasets have enough data for training. The only drawback is that because the isolated nodes never receive any information from other nodes, they will be more conservative in their response to changes in the data, with the risk of potentially higher blind spots in the individual datasets.

The impact of adding new workers

To facilitate the adding of new workers at a later time, information about the current round must be maintained in the message exchange between the master and the workers. When an FL task starts, all workers register to round ID 0, which triggers the master to initialize the random weights and broadcast the same distribution to all workers. All workers train in parallel and contribute to the same training round. As the rounds increase, the federated model’s maturity increases until a saturation point is reached.

If the current round ID is greater than 0, the master is aware that the process of averaging of weights has taken place at least once, which means that the model is not at a random initial state. When a new worker joins the FL task, it sends its round ID as 0.

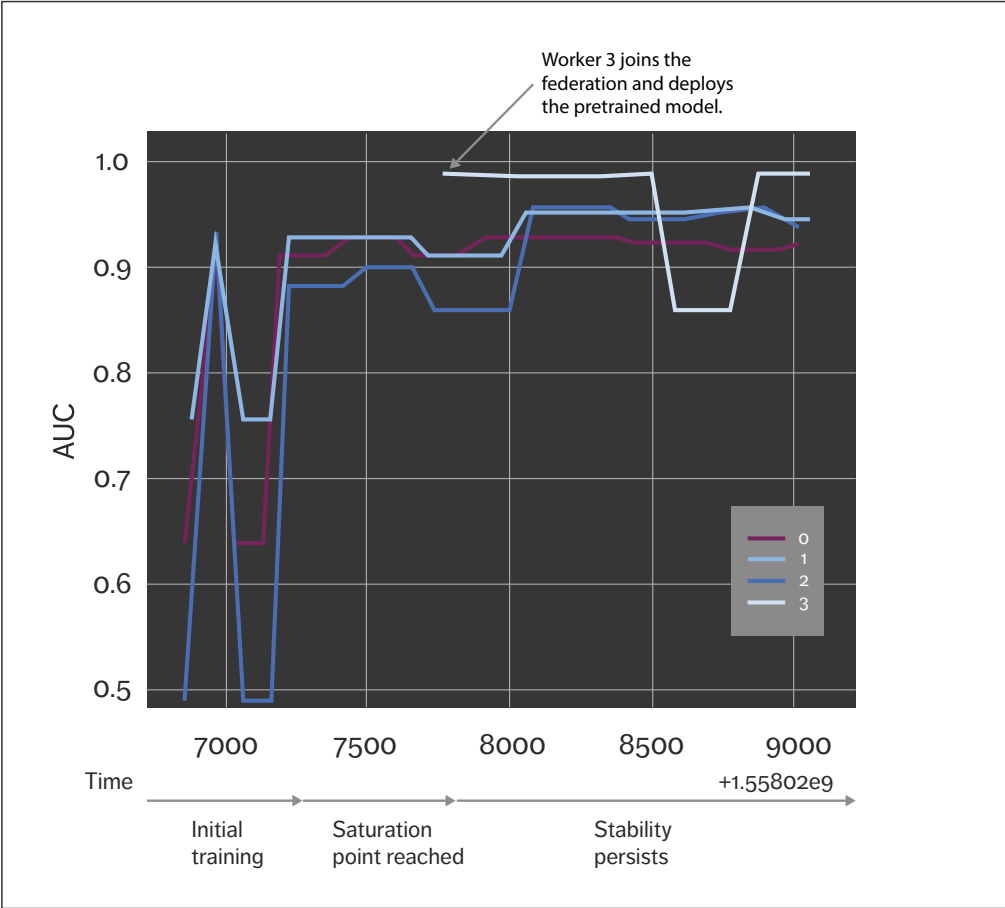


Figure 4 AUC scores before and after a new worker joins the FL task

The master, whose latest round ID is greater than 0, recognizes the worker as new and immediately shares the latest state of the model with the new worker after the first handshake.

Figure 4 illustrates how accuracy persists in the FL model when a new worker joins. In this example, three workers numbered 0, 1 and 2 contribute to the initial training phase of the model, receive the same randomized weight matrices from the master node, and train on the same model. As time passes, accuracy reaches a saturation point. Later, worker 3

joins the FL task. However, since worker 3 is at its first round, it is allowed to use the saturated model, but not allowed to contribute in the weight aggregation. Instead, its weights are discarded.

Network footprint comparison

Table C in Figure 3 presents a set of formulas that can be used to determine the network footprint for centralized ML, isolated ML and FL scenarios. When training an ML model in a conventional way (as in centralized ML), it is assumed that the number

of message exchanges is equal to the number of regions where the compressed version of the dataset is transferred – that is, the number of local deployments.

In FL, on the other hand, there are multiple message exchanges that consist of rounds, which refers to the number of training phases required until the accuracy of the model converges. Message size in FL is determined by the serialized vector that contains the neural weights, which is directly related to the size of the neural network.

Table D in Figure 3 presents the network footprint results for the preventive maintenance use case, showing that the FL approach yielded a one order of magnitude reduction in data volume compared with the centralized approach – a drop from 2,000MB to 19.2MB. This dramatic reduction can be attributed to the simplicity of the neural network and the substantially smaller amount of data that needs to be transferred in the FL scenario. In the long term, this significantly smaller network footprint will enable the creation of a more complex neural network with the ability to detect more complex patterns in the dataset.

Conclusion

While conventional machine learning (ML) models provide many benefits to mobile network operators, particularly in terms of ensuring consistent QoE, the large data transfer that they require results in a substantial network footprint and can lead to privacy issues. By bringing “the computation to the data” instead of transferring “the data to the computation,” federated learning (FL) makes it possible to overcome those challenges by training a centralized model on decentralized data.

Ericsson’s research in this area has demonstrated that it is possible to migrate from a conventional ML model (trained using a completely centralized dataset) to a federated one, significantly reducing data transfer and protecting privacy while achieving on-par accuracy. In light of our findings, we believe that FL has an important role to play in the ongoing automation of the telecom sector and in the transition to the zero-touch networks of the future.

References

1. Communication-Efficient Learning of Deep Networks from Decentralized Data, February 28, 2017, H. Brendan McMahan et al., available at: <https://arxiv.org/pdf/1602.05629.pdf>
2. Google, Practical Secure Aggregation for Privacy-Preserving Machine Learning, 2017, K. Bonawitz et al., available at: <https://ai.google/research/pubs/pub47246/>
3. Deep Learning with Differential Privacy, October 25, 2016, M. Abadi et al., available at: <https://arxiv.org/pdf/1607.00133.pdf>
4. Distributed Systems: Principles and Paradigms, 2002, A. Tanenbaum et al., available at: <https://www.amazon.com/Distributed-Systems-Principles-Andrew-Tanenbaum/dp/0130888931>

Further reading

» Privacy Preserving QoE Modeling using Collaborative Learning, October 21, 2019, Selim Ickin, Konstantinos Vandikas, and Markus Fiedler, in the 4th Internet-QoE Workshop: QoE-based Analysis and Management of Data Communication Networks (Internet-QoE’19), ACM, New York, NY, USA, available at: <https://doi.org/10.1145/3349611.3355548>

THE AUTHORS



Konstantinos Vandikas

◆ is a principal researcher at Ericsson Research whose work focuses on the intersection between distributed systems and AI. His background is in distributed systems and service-oriented architectures. He has been with Ericsson Research since 2007, actively evolving research concepts from inception to commercialization. Vandikas has 23 granted patents and over 60 patent applications. He has authored or coauthored more than 20 scientific publications and has participated in technical committees at several conferences in the areas of cloud computing, the Internet of Things and AI. He holds a Ph.D. in computer science from RWTH Aachen University, Germany.

Selim Ickin

◆ joined Ericsson Research in 2014 and is currently

a senior researcher in the AI department in Sweden. His work has been mostly around building ML prototypes in diverse domains such as to improve network-based video streaming performance, to reduce subscriber churn rate for a video service provider and to reduce network operation cost. He holds a B.Sc. in electrical and



electronics engineering from Bilkent University in Ankara, Turkey, as well as an M.Sc. and a Ph.D. in computing from Blekinge Institute of Technology in Sweden. He has authored or coauthored more than 20 publications since 2010. He also has patents in the area of ML within the scope of radio network applications.

Gaurav Dixit

◆ joined Ericsson in 2012. He currently heads the Automation and AI Development function for Business Area Managed

Services. In earlier roles he was a member of the team that set up the cloud product



business within Ericsson. He holds an MBA from the Indian Institute of Management in Lucknow, India, and the Università Bocconi in Milan, Italy, as well as a B.Tech. in electronics and communication engineering from the National Institute of Technology in Jalandhar, India.



Michael Buisman

◆ is a strategic systems director at Business Area Managed Services whose work focuses on ML and AI. He joined Ericsson in 2007 and has more than 20 years of experience of delivering

new innovations in the telecom industry that drive the transition to a digital world. For the past two years, Buisman and his team have been developing a managed services ML/AI solution that is now being deployed to several customers globally. Buisman holds a BA from the University of Portsmouth in the UK and an MBA from St. Joseph's University in Philadelphia in the US.



Jonas Åkeson

◆ joined Ericsson in 2005. In his current role, he drives the implementation of AI and automation in the three areas that integrate Ericsson's Managed Services business. He holds an M.Sc. in engineering from Linköping Institute of Technology, Sweden, and a higher education diploma in business economics from Stockholm University, Sweden.



Cognitive processes for adaptive intent-based networking

Autonomously operated and self-adapting networks will make it possible to utilize the capabilities of 5G networks in new business models and achieve an unprecedented level of efficiency in service delivery. Intents will play a critical role in achieving this zero-touch vision, serving as the mechanism that formally specifies what the autonomous system is expected to do.

JÖRG NIEMÖLLER,
LEONID MOKRUSHIN,
SWARUP KUMAR
MOHALIK,
MARTHA VLACHOU-
KONCHYLAKI,
GEORGE SARMONIKAS

5G networks introduce unprecedented flexibility and dynamic adaptation into service delivery and network resource utilization. In the business layer, this is reflected in the ability to offer customizable service products with detailed agreements on functional and non-functional characteristics as well as fast delivery. Dynamic adaptation to changes within the constraints of stringent requirements on lead and reaction times is beyond the capacity of a human workforce. Extensive automation will be necessary to overcome this challenge.

■ The zero-touch paradigm implies that the operation of services and the underlying networks is autonomous and does not require human intervention. To achieve this, the zero-touch system must be able to handle the complexity caused by continuous changes to the system at the same time that it delivers services to users and manages issues such as the cost of resources versus the budget available, the legal compliance of the service and the security of the setup. This is challenging for technical systems in real-world scenarios.

For example, successful service operation requires each service to be properly provisioned and assured to deliver the promised function with agreed

performance metrics. Complexity arises as a result of changes to contracts, products, customer preferences, the business strategy or the environment in which the service is being offered. Users may start exhibiting new behavior, leading to varying service usage patterns and network loads, or the network may change due to upgrades, reconfiguration or outages. Some changes may be regular and predictable, whereas others are sudden and surprising.

To manage all of these concerns autonomously and adapt its behavior appropriately, a zero-touch system must understand every aspect of what is expected of it. Each requirement and goal must be carefully defined in order for technical processes to derive suitable and optimized actions to manage it. These definitions are known as intents.

From the perspective of a human operator, an intent expresses the expectation of what the operational system is supposed to deliver and how it behaves. In light of this, we define intent as “formal specification of all expectations including requirements, goals and constraints given to a technical system.”

The role of intents in cognitive networks

Everything an autonomous system needs to know about its goals and expected behavior must be defined with intents. The system will not perform any operation unless it relates to the fulfillment and assurance of an intent, which means that all goals – including those that may have been considered “common sense” in human-operated systems – must be expressed as intents.

An intent in an autonomous system is ideally expressed declaratively – that is, as a utility-level goal that describes the properties of a satisfactory outcome rather than prescribing a specific solution. This gives the system the flexibility to explore various solution options and find the optimal one. It also allows the system to optimize by choosing its own goals that maximize utility.

Unlike traditional software systems, where requirements are analyzed offline to detect and resolve conflicts prior to implementation, intents are added to an autonomous system during runtime. Adaptation to changed intent as well as conflict detection and resolution are therefore essential capabilities of an autonomous system.

One of the benefits of expressing intents as utility-level goals is that it helps the system cope with the conflicting objectives of multiple intents. This is vital, because an autonomous system often has to take multiple intents into account before making a decision.

For example, an autonomous system may have one intent to deliver a service with high QoE, while another may be to minimize resource spending. It can resolve such conflicts either explicitly from weights that introduce relative importance or implicitly from properties of preferential outcomes as defined in utility-level goals.

Expectations originate from contracts or business strategy and remain constant when the underlying system is replaced or modified. Consequently, when setting up the intents, it is important that they are formulated in an infrastructure-agnostic way, so that they can be transferred across system generations and implementations.

Terms and abbreviations

AI – Artificial Intelligence | BSS – Business Support Systems | CEM – Customer Experience Management | IoT – Internet of Things | KPI – Key Performance Indicator | OSS – Operations Support Systems | RDF – Resource Description Framework | SLA – Service Level Agreement | SLO – Service Level Objective | SLS – Service Level Specification | SON – Self-Organizing Networks | TOSCA – Topology and Orchestration Specification for Cloud Applications | VNF – Virtual Network Function

In short, the intent establishes a universal mechanism for defining expectations for different layers of network operation. It expresses goals, utility, requirements and constraints. It defines expectations on service delivery as well as the behavior of the autonomous operational system and the underlying network.

●● [THE INTENT] EXPRESSES GOALS, UTILITY, REQUIREMENTS AND CONSTRAINTS ●●

Service-specific intents

One essential type of intent relates to the specification of services. Service-specific intents state expected functional and performance characteristics. Service Level Agreements (SLAs), Service Level Specifications (SLs), Service Level Objectives (SLOs) and TOSCA (Topology and Orchestration Specification for Cloud Applications) [1] models are all examples of service-specific intents that are used on different levels in the operations stack.

SLAs are business support systems (BSS) objects. Service-specific intents based on SLAs specify the promised service and include expected performance details and business consequences such as payment for delivery and penalties when failing.

SLs/SLOs define the service delivery details at operations support systems (OSS) level. Based on this input, autonomous OSS would plan detailed tasks to realize the service delivery. TOSCA models would be used to express further technical details the OSS generate (expectations from orchestration and assurance).

In multiple stages, the autonomous operation makes decisions about further details. Higher-level intent is the input leading to the lower-level intent that is used to distribute specific goals to subsystems. For example, SLAs/SLs are the intents that express a terminal goal of the OSS. The OSS then decide which TOSCA model would be the best option to deliver the promised performance with minimal resource usage. The selected TOSCA

model is the instrumental goal of OSS and becomes an intent and terminal goal for the orchestrator. This pattern of making decisions based on a given intent and taking action by sending lower-level intents to subsystems is the key interwork mechanism of intent-based operation, according to which the entire operations stack of autonomous networks is built.

Strategic and behavioral intents

Beyond all the service-specific intents that an autonomous system must have, it also requires guidance on how to handle strategic and behavioral concerns. Traditionally implemented in the form of manually coded policies, this type of guidance steers general system behavior and supports the type of decision-making that has traditionally been based on human intuition and experience, along with knowledge about context and operator strategy. Intent-based operation makes it possible for operators that want to handle these concerns in a more dynamic fashion to replace manually coded policies with strategic and behavioral intents.

This is useful in cases where the operator chooses to require a default minimum security level that differs from that which is implemented into the service, for example. In these cases, dedicated intent can be used to set the security level for all services that do not specify it directly.

With regard to legal compliance, services may be delivered in multiple markets where different rules apply. A legal-compliance intent requires compliance and potentially specifies the details.

Since there is always a risk of service degradation when changes are initiated and risky actions may sometimes lead to a higher margin, risk-management intents can be used to convey how the operator wants the autonomous system to balance risks versus potential gain.

Reporting/escalation intents steer how the autonomous system interacts with the human workforce by reporting progress status on intent fulfillment and seeking manual decision in escalations.

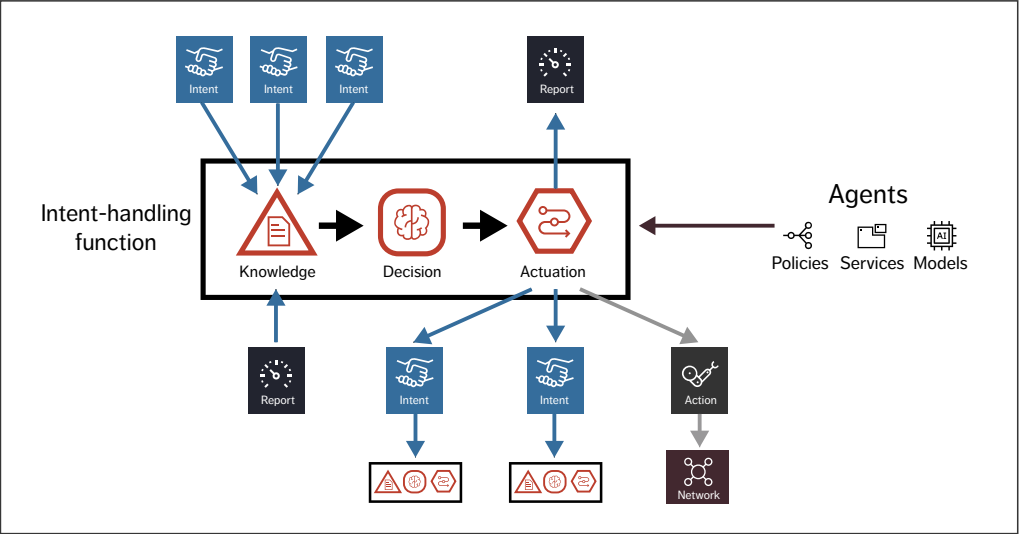


Figure 1 The intent-handling function

Formally expressing an intent

An autonomous system requires intents to be formally defined in a machine-readable and processable way, but the broad range of considerations involved and their abstract semantics are often difficult to structure. Techniques from knowledge management and semantic modeling enable the creation of an ontology of intent, based on an extensible metamodel. Resource Description Framework (RDF) [2] and RDF Schema [3] standards can be used for knowledge modeling.

Technical functions such as contract and order management would directly use RDF objects to communicate intent. Intent specified directly by human operators would require an intuitive frontend, potentially using natural language.

Intent handling

The operation of services within an intent-based network also requires the introduction of intent-handling functions in the operations stack and functional architecture. An intent-handling function receives the intents, decides which

actions must be taken to optimally fulfill all given intents and implements its decisions.

Intent-handling functions have a knowledge base that contains the intent ontology. They also have machine-reasoning capabilities to realize knowledge-driven decision-making processes.

Machine reasoning plays a key role in intent handling, with its capability to understand abstract concepts from diverse domains and provide precise, specialized conclusions based on precedent and observation. Probabilistic modeling contributes quantification of risk and uncertainty, which is essential to make informed decisions when facing conflicting goals and new situations.

Figure 1 shows how the intent-handling function works. While its implementation is domain-specific, its interface is generic. It receives intents that express all types of expectations. It is equipped with policies and artificial intelligence (AI) models that implement the capabilities needed for analyzing the system state and finding optimized operational actions based on observations from the operated environment. The intent handler also reports the fulfillment and assurance status of its intents.

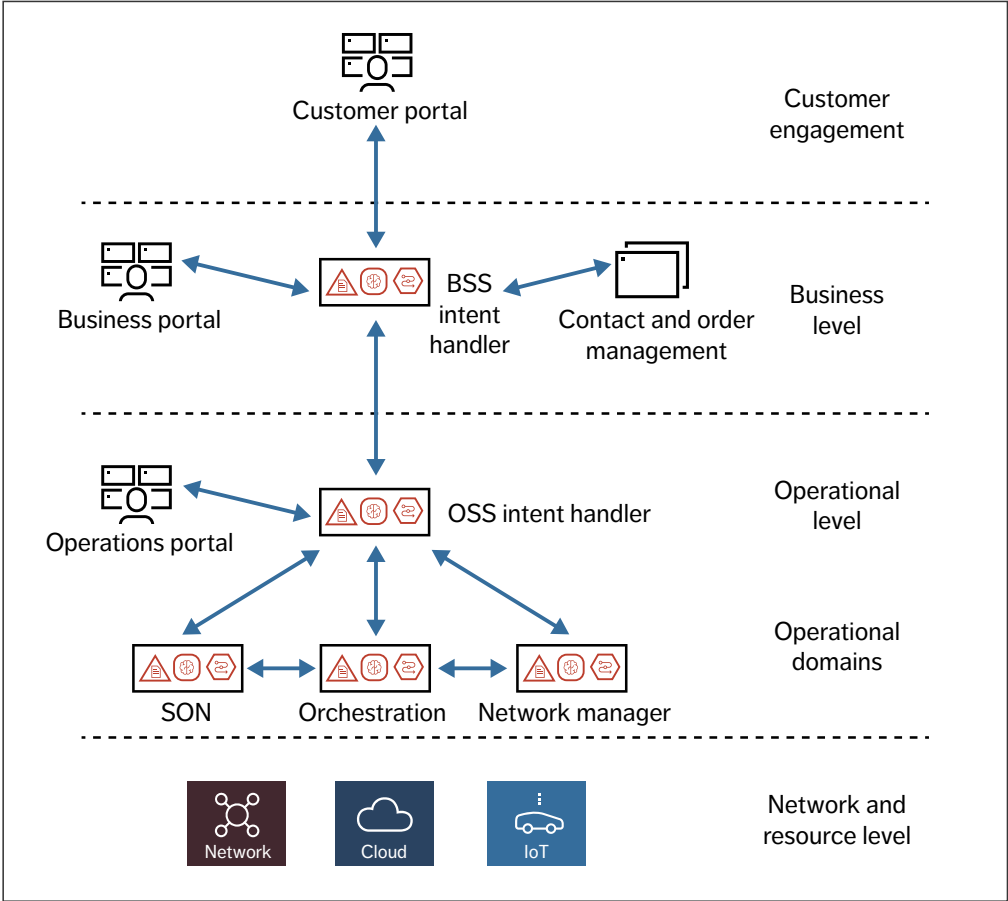


Figure 2 Intent-driven multi-layer operations system

The API (application programming interface) of the intent-handling function is domain-independent. Its main objective is to manage the life cycle of intents. It implements methods to set, modify and remove intents and send reports. Intent is constructed based on a common intent meta-model and its details are specified according to domain-specific information models. Intent management is therefore primarily knowledge management.

Figure 2 provides an example of how intent-handling functions can be combined to realize

a complete intent-driven operations system. Every major system layer and subsystem domain, including BSS, OSS, orchestration and network management, contains an intent-handling function. Intent originates from functions such as contract and order management. Additional intents can be entered directly through portals.

Introducing the cognitive layer

Lexico defines cognition as: "... the mental action or process of acquiring knowledge

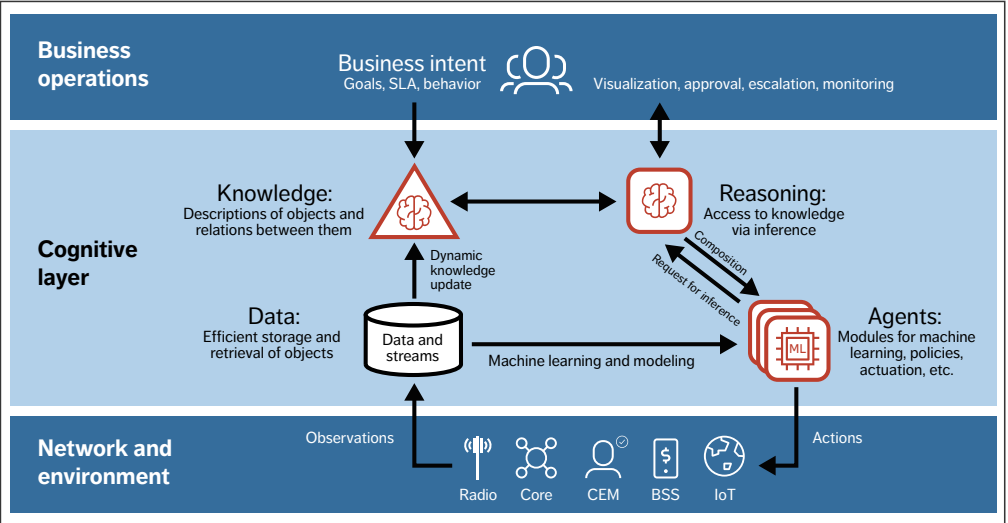


Figure 3 Functional architecture of the cognitive layer

and understanding through experience and the senses.” [4] As it is designed to perform the equivalent operational tasks of understanding through experiencing and sensing, an autonomous system is, therefore, a technical implementation of cognition.

Creating an intent-handling function that understands complex and abstract intent semantics, derives the optimal target state and plans actions for transitioning the system into this state is a challenging task. The function must be able to explore options, learn from precedents and assess the feasibility of actions based on their expected consequences.

By combining well-understood AI techniques within a flexible architecture, we have designed a cognitive system that specializes in autonomous service and network operation. We refer to it as the cognitive layer, and its role is to serve as an interface between business operations and the network/ environment, as shown in Figure 3.

The cognitive layer consists of three essential components: a knowledge base, a reasoning engine and an agent architecture. The knowledge base

contains the ontology of intents along with domain-specific knowledge such as the current state of the system. The domain-independent reasoning engine uses the knowledge graph and serves as the central coordinator function for finding actions, evaluating their impact and ordering their execution. Finally, the agent architecture allows any number of models and services to be used. Agents can contain machine-learned models or rule-based policies, or implement services needed in the cognitive reasoning process.

To be usable, an agent needs to be registered and described in the knowledge base. Its description can be added and modified at any time, allowing life cycles of the models, policies and supplementary services to be decoupled from the overall life cycle of the cognitive layer.

The agent metadata contains a description of the agent interface, along with its function, role and capabilities. For example, we have implemented a machine-learned model that can propose radio base station configurations that optimize the service experience. This model is registered as an agent in the role of a “proposer” for configuration actions.

The separate life cycle makes it possible for the model to be replaced with an improved version when available, independent of the cognitive layer release cycles.

We have also demonstrated agents in the role of “predictor” with the ability to estimate the effect of actions on key performance indicators (KPIs). An agent in an “observer” role would monitor data sources, keeping knowledge about the state up-to-date. An agent in the “actuator” role can implement actions in the network by utilizing, for example, established network management functions.

●● SPECIALIST AGENTS ARE USED INTENSIVELY IN EVERY STEP OF THE PROCESS ●●

How the cognitive layer works

The successful operation of the cognitive layer depends on smooth interaction between the reasoning engine and the knowledge base. The reasoning engine continuously executes a process that tries to find actions to close the gap between the current observed state and the wanted state, according to the intent. It collects proposals, obtains predictions on the effect of each proposal, evaluates gain versus risk and certainty, prioritizes actions and executes its decisions. Specialist agents are used intensively in every step of the process.

The reasoning engine is an adaptive knowledge-driven composer that can instantiate the cognitive process following changes in intent, state and context. It can dynamically compose specialized agents and add them into the process if their capabilities and roles match the needs according to intent and context. This is, for example, how the cognitive layer obtains action proposals from agents that are implementing suitable models.

In cases where the capabilities of multiple agents match the requirements of a role, each of them can be used simultaneously to generate alternative

solution strategies, resulting in a diverse set of options for the prediction and evaluation steps that follow. This coexistence of agents makes it possible to combine rule- and policy-based implementations with machine-learned alternatives in the same system, enabling the system to acquire new advanced abilities without losing current ones.

The cognitive process is a perpetual loop that starts again directly after the previous iteration has finished. Any degradation of the network or issues with services would be visible in the observed state. By trying to close the gap to the wanted state set by intent, the cognitive layer implicitly addresses incidents. Even without explicit issues, the continuous cognitive process would still seek actions for further optimization. It could, for example, try to deliver the same services with reduced resources.

Through its reasoning-based core process, the cognitive layer reaches a high degree of dynamic adaptability to new situations. This is in stark contrast to systems that have been realized through rule-based policies and fixed workflows, where every supported situation needs consideration at design time through suitable branches in the decision tree and diversifying rules. Existing rule-based policies can, however, still be used in the cognitive layer through integration as agents. This opens an upgrade path from legacy automation, with AI-based models added gradually.

Example use case

To demonstrate how the cognitive layer works, we have experimented with a use case that optimizes the provisioning decisions for the deployment of a virtual network function (VNF). The source of the intent is an SLA that specifies the service to be delivered along with the KPI targets that have been promised to the customer. Our example use case requires that the VNF be deployed with strict targets on latency and throughput.

We started by developing the required proposer and predictor agents in an offline data science process. We then deployed the service in a test environment and exposed it to a range of usage loads. We also explored deployment options by varying

parameters of the underlying TOSCA model. Combined with measured KPIs, these variations create training data sets for learning a model that is capable of connecting deployment options to expected performance. This exercise created a proposal agent that is able to recommend a TOSCA model configuration optimized for the latency and throughput figures required by the SLA.

In our experiment, online fulfillment – from receipt of the SLA to optimized deployment – is fully autonomous and controlled by the reasoning process of the cognitive layer. When the intent derived from the SLA arrives in the knowledge base, the reasoning loop starts processing it. The reasoner finds that the agent learned in the test environment matches the needs and requests a proposal. The agent delivers a TOSCA model fully configured and optimized for the requested KPI target and proposes to deploy accordingly.

The cognitive layer then uses prediction and evaluation agents to assess the proposal. In this case, our prediction agent contains a state-action model – a probabilistic graph based on Markov decision process modeling. The model is continuously learned from observing states and the results of observed actions. It has the ability to estimate the probability of expected result states for proposed actions, enabling informed decisions about actions considering their risk.

In the evaluation step, the autonomous system detects and resolves all remaining conflicts between intents. It also decides on escalations if the risk of the proposed deployment or uncertainty of the models is too high. If escalation is required, the cognitive process requests the support of a human technician, presents the situation including proposals and predictions, and asks for approval.

While the cognitive layer can operate fully autonomously, it knows when the human workforce wants to be involved. The exact threshold for escalation is at the discretion of the operator and is determined by behavioral intent. If the system gains human trust as a result of presenting many good actions, the threshold for fully autonomous decisions can be lowered.

When a proposed action is selected after verifying that the impact on all intents is advantageous and risk is reasonably low, it is then handed over for execution to the orchestrator behind the actuation agent. This concludes the first intent-handling loop after the new intent for the new SLA was introduced to the cognitive layer. The gap between the new intent and the current state was particularly wide since the new service was not yet provisioned. This gap narrowed through a provisioning action. Further iterations continuously monitor the service deployment, optimize it when possible or heal when needed. In this way, the continuous cognitive operation process provides fulfillment and assurance of expectations formulated as intents.

●● ONLINE FULFILLMENT IS FULLY AUTONOMOUS AND CONTROLLED BY THE REASONING PROCESS ●●

Conclusion

Zero-touch autonomy is an ideal beyond reach as long as artificial intelligence (AI) cannot match human capability to reason and decide within complex dependencies and broad domains. However, by using a combination of currently available and well-understood AI techniques within a flexible architecture, it is possible to reach a high degree of practical autonomous operation.

Our use case example demonstrates how the cognitive layer that we have developed can enable autonomous operation with the use of intents. It is built with an agent architecture that decouples the life cycles of the agents. It is coordinated by knowledge-driven reasoning that composes machine-learned models and legacy implementations. It evaluates action impact on intent fulfillment and makes decisions based on expected gains and risks, while resolving conflicting goals and maximizing utility. It can adapt to new situations through learning. The resulting system can execute many of the cognitive considerations human technicians

make when they operate services and networks manually. However, the cognitive layer does this with continuous attention and near-immediate reaction.

The adaptive reasoning capabilities of the cognitive layer enable effective management of growing network complexity, dramatically reducing

the need for humans to manually modify policies or participate in online decision-making. This frees up the human workforce to concentrate on offline tasks such as executing data science processes, optimizing the available models and defining business strategies that are implemented by intent setting.

References

- 1. **OASIS, TOSCA Version 2.0, Committee Specification Draft 02, June 25, 2020, available at:** <http://docs.oasis-open.org/tosca/TOSCA/v2.0/TOSCA-v2.0.html>
- 2. **W3C, RDF 1.1 Concepts and Abstract Syntax, W3C Recommendation, February 25, 2014, available at:** <https://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/>
- 3. **W3C, RDF Schema 1.1, W3C Recommendation, February 25, 2014, available at:** <https://www.w3.org/TR/rdf-schema/>
- 4. **Cognition, Lexico.com, available at:** <https://www.lexico.com/definition/cognition>

Further reading

- » **Ericsson Technology Review, Cognitive Technologies in Network and Business Automation, 2018, Niemöller J; Mokrushin, L, available at:** <https://www.ericsson.com/en/reports-and-papers/ericsson-technology-review/articles/cognitive-technologies-in-network-and-business-automation>
- » **Ericsson, AI by Design, available at:** <https://www.ericsson.com/en/ai-and-automation>

THE AUTHORS



Jörg Niemöller

◆ is an expert in analytics and customer experience. He joined Ericsson in 1998 and has since held multiple positions in research as well as in system management for core network and digital services. His current focus is innovation in OSS through architecture and solutions for autonomous network and service operation. Niemöller holds a Ph.D. in computer science from Tilburg University, the Netherlands, and a diploma degree in electrical engineering from the TU Dortmund University, Germany.

Leonid Mokrushin

◆ is a principal researcher at Ericsson Research. With a background in computer science and formal methods, he is currently focusing on knowledge-intensive symbolic AI systems and their practical applications



Swarup Kumar Mohalik

◆ is a principal researcher at Ericsson Research who joined the company in 2015. His expertise is in the areas of AI and formal methods, and his work primarily



in the telecom domain. Mokrushin joined Ericsson in 2007 after postgraduate studies at Uppsala University in Sweden, where he specialized in formal verification of real-time focuses on applying them to service automatization and the Internet of Things (IoT). He has research experience in the areas of formal specification and verification of real-time embedded software and AI planning techniques. Mohalik holds a Ph.D. in computer science from the Institute of Mathematical Sciences, Chennai, India, and a post-doctoral fellowship at LaBRI, University of Bordeaux, France.



Martha Vlachou-Konchylaki

◆ is a director of technology strategy specializing in AI and data strategy. She joined Ericsson in 2015 and has been working within different groups within Ericsson, from machine learning prototyping and business development to AI strategy. Vlachou-Konchylaki holds an M.S. in machine learning from the KTH Royal Institute

of Technology in Stockholm, Sweden, and a B.Eng. in electrical and computer engineering from the University of Patras, Greece.

George Sarmonikas

◆ joined Ericsson in 2013 after several years of working for mobile operators. He currently leads AI business innovation within Digital Services. Prior to this, he was responsible for product management of Ericsson's customer experience management (CEM) and analytics portfolio, including assets for subjective experience scoring. Sarmonikas holds both an M.Sc. in communication systems and an M.Eng. in electronic engineering and computer science from the University of Bristol in the UK, as well as a graduate diploma in artificial intelligence from Stanford University, US.



Data ingestion architecture for telecom applications

A fast-growing number of business processes and applications rely on data-driven decision-making and telecom artificial intelligence (AI). As new technologies emerge that make it possible to extract additional value from collected data, the opportunities for enhanced analytics and data-driven, AI-powered decision-making continue to expand. To fully capitalize on collected data, communication service providers need data pipelines that are based on a harmonized data ingestion architecture.

ANNA-KARIN
RÖNNBERG,
BO ÅSTRÖM,
BULENT GECER

The purpose of data pipelines is to facilitate access to high-quality data for applications ranging from network management and customer experience management (CEM) to business analytics, product serviceability, artificial intelligence (AI)/machine learning (ML) model training, tailored service offerings and AI for Internet of Things solutions. As many of these applications use the same data sets, a harmonized data ingestion architecture boosts efficiency and makes it possible to focus application development resources on use case realizations rather than on data management.

■ Our harmonized data ingestion architecture is built on the idea that data should be collected once and then shared with any authorized application that needs it. The architecture we propose can be deployed both in customer networks and as a service in application clusters (ACs) external to customer networks. It handles data in accordance with customer contracts using automated procedures and provides security mechanisms with controlled access to original data and de-identified data sets.

Data discovery and the ability to control quality and data life cycle are inherent capabilities of the harmonized data ingestion architecture that build trustworthiness for the benefit of every application that consumes the data. Compliance with standards

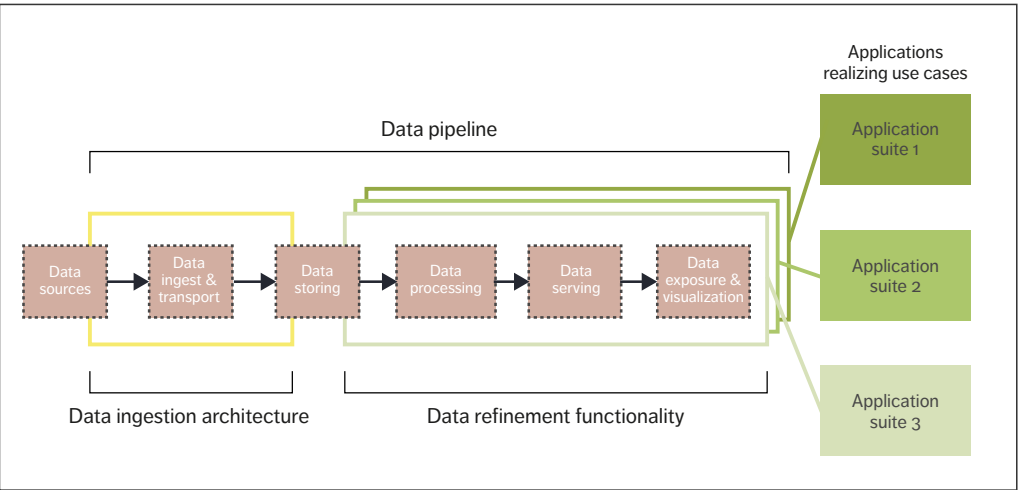


Figure 1 Data pipeline using the harmonized data ingestion architecture

and compatibility with evolving architectures, including ONAP (the Open Network Automation Platform) and ORAN (the Open Radio Access Network), are of crucial importance.

Data pipeline functionality areas

As shown in Figure 1, a data pipeline consists of two main functionality areas: data ingestion and data refinement. One conventional approach to data pipelines has been for each application suite to have its own data pipeline, with its own data ingestion and data refinement functionality. To overcome the inefficiencies of this approach, our goal has been to create a harmonized solution for data ingestion in which any collected data is made available for

further processing in data-refinement functionality and data-consuming applications. In other words, rather than locking data into one data pipeline or application, we want to make it available to any application suite and its associated data pipeline.

The colored lines in Figure 1 show how the common sets of data collected and made available by the data ingestion functionality (marked in yellow) can be refined in several ways (marked in shades of green) to facilitate and tailor the usage of data in different application suites. At Ericsson, we refer to this approach as “democratizing data.”

The benefits of our novel approach to data ingestion include significant opex and capex reductions, increased average revenue per user

Terms and abbreviations

AC – Application Cluster | AI – Artificial Intelligence | ARPU – Average Revenue Per User | BDR – Bulk Data Repository | CEM – Customer Experience Management | DCC – Data Collection Controller | DDC – Data Distribution Central | DR&D – Data Routing & Distribution | DRG – Data Relay Gateway | DWH – Data Warehouse | EDCA – Extensible Data Collection Architecture | GDC – Global Data Catalog | ML – Machine Learning | ONAP – Open Network Automation Platform

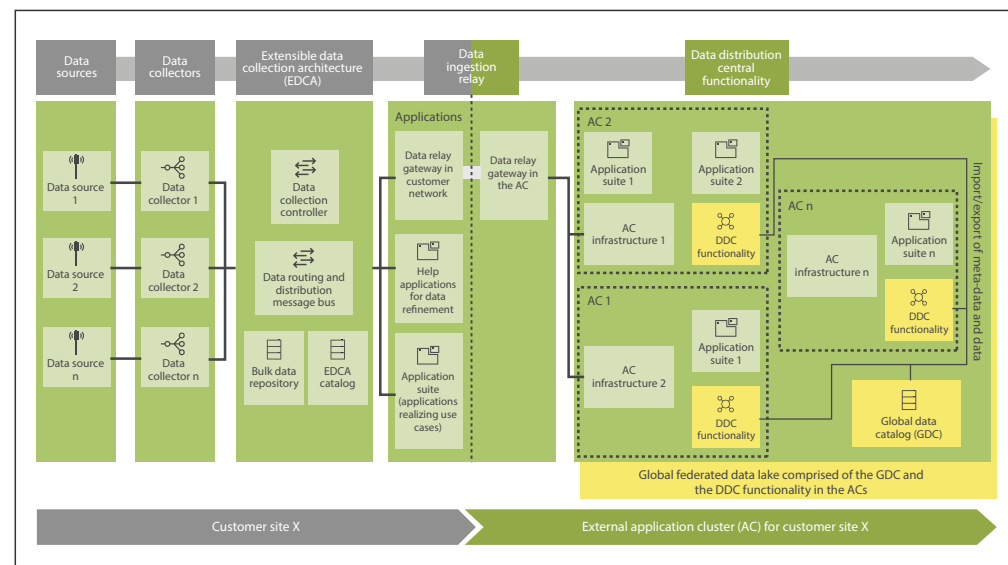


Figure 2 Data ingestion architecture

(ARPU), business process refinement and proactive product support. Opex reductions result from the automation of network assurance, while more efficient network planning and optimization lead to capex savings. The increase in ARPU is due to improvements to network performance and better CEM. Business process refinement comes from increased data quality and faster access to collected data, with the ability to use data collected in near-real time with historical trend analysis. The benefits in terms of proactive product support come from the automation of life-cycle management and serviceability combined with AI-powered decision-making.

Harmonized data ingestion architecture

Put simply, the main objective of our data ingestion architecture is to securely collect data once from the data sources and allow many different consuming applications to use the data in accordance with privacy regulations and customer contracts. These applications may reside in customer networks or be provided as a service and hosted outside the

customer networks in application clusters on Ericsson premises or in public clouds, for example.

Figure 2 illustrates the example of a customer network that includes both applications at the customer site (on the left) and applications provided as a service (on the right) as ACs, which are sites where application suites are deployed, typically for a particular purpose such as network operations or product support. Each AC consists of infrastructure and applications where the infrastructure may or may not be cloud based.

Data can be collected from sources for consumption by any application and use case. The collected data is then made available to all applications at the customer site, such as helper applications for data refinement and applications that realize use cases. In accordance with customer contracts, the data ingestion relay transfers data from the customer network to the ACs in Ericsson Cloud using the data relay gateway (DRG). With the help of data distribution central (DDC) functionality and the global data catalog (GDC), the ACs cooperate and form a global federated data lake.

Key functional entities in the data ingestion architecture

The key functional entities in the data ingestion architecture are data sources, data collectors, the extensible data collection architecture (EDCA), the DRG, the DDC functionality, the GDC and the global federated data lake.

Data sources

A data source is a generic term that refers to any type of entity that exports data, such as a node in a telecom network, cloud infrastructure components, a social media site or a weather site. Data sources may export several types of data – including data about performance management, fault management or configuration management – as well as logs and/or statistics from nodes in a telecom network. Data can be exported either by streaming events, files or both.

Data export interfaces will in many cases follow established standards but may also be proprietary. The data can either be pushed from the data source to a data collector or pulled from the data sources by data collectors.

A data source should allow configuration of the data export characteristics, so that it is possible to increase or decrease the data collection frequency, for example. It is not advisable from a load perspective to always collect the maximum number of events from a data source.

From a security point of view, data sources and data collectors should use two-way authentication and encrypt the exported data.

Data collectors

Data collectors facilitate data collection from data sources by using their data export capabilities. They are designed to cause minimal impact to the performance of the data sources. The procedures should be optimized from a data source perspective, and data sets should only be collected once instead of repeatedly. Data can be collected as real-time events or as files in batches depending on the type of data and use cases.

Data collectors make the data available on the EDCA for consumption by multiple applications. It

is easy to avoid collecting data that is already available by first checking the EDCA catalog to see which data sets already exist before deploying a new data collector. In this way, data collectors register metadata about their collected data in the EDCA catalog, publish data on data routing and distribution (DR&D) and optionally persist data in the bulk data repository (BDR). Notifications of collected files stored in the BDR are published on DR&D to any consuming application that subscribes to the collected files. The notifications contain references and an address to the BDR that stores the file.

The data collectors collect raw data, and there is an option to apply filtering on the raw data before it is published on DR&D. Further refinement of data can then be tailored for different data pipelines and application suites by EDCA helper applications that implement data-processing and data-serving pipeline functionality, making data ready for visualization and exposure.

Data collectors and EDCA helper applications plug into the EDCA and will evolve according to use case requirements on data pipelines to serve the application suites. It is also possible to integrate data collection from other vendors' data sources with the EDCA.

Extensible data collection architecture

The DR&D function in the EDCA is tasked with receiving data publications from data collectors and delivering these to the consuming applications that subscribe to them. Data on DR&D can be streaming events or notifications of files persisted in the EDCA's BDR. Applications that process the data they receive from DR&D can publish their insights (processed data) back to DR&D, so other applications can benefit from them.

The message bus functionality in DR&D can be realized with one or more message bus technologies, such as Apache Kafka for ONAP compliance, but DR&D is not limited to Kafka. Abstractions are used to shield producers and consumers from the message bus technologies. The DMaap (Data Movement as a Platform) service wrappers in ONAP are one example of such an abstraction.

DATA SCHEMAS CAN BE USED TO DYNAMICALLY LOAD MODELS INTO CONSUMING MODEL-DRIVEN APPLICATIONS

DR&D primarily provides the functionality to publish, subscribe to and deliver data, but it can also persist real-time streaming events published on DR&D to enable historical analytics.

DR&D is agnostic to the format of the transferred data. The EDCA catalog stores references to metadata specifications for data sets published on DR&D to facilitate the interpretation and consumption of data by applications. Metadata specifications contain the data schemas (models), and these can be used to dynamically load models into consuming model-driven applications.

The EDCA catalog enables data consumers to discover available data, with instructions on how to access it for consumption. Data must be interpreted to be consumed, and for this purpose the catalog also contains references to metadata specifications for data sets available on the EDCA. Examples of the kind of metadata stored in the catalog include:

- » type of data, such as performance management data
- » class of data (raw data or insight)
- » source of origin
- » data quality indication
- » data lineage information, including all transformations that have occurred
- » location
- » data (set) reference
- » logical name (URL) for data consumption (DR&D topic)
- » schema and serialization method, such as Avro
- » metadata specification references, such as references to schemas stored in a schema repository.

The BDR provides the capability to store large amounts of structured, semi-structured and unstructured data – that is, data sometimes referred to as big data. The BDR exposes an (application programming) interface to allow consuming applications that have been notified on DR&D to retrieve a file by using the address and reference included in the notification.

Note that the BDR is not a traditional data warehouse (DWH) intended for long-term data storage. If consuming applications need longer-term storage, subsets of the data should be copied from the BDR and stored in application-specific, long-term DWHs.

The data collection controller (DCC) plays a notable role in the EDCA. Its purpose is to instruct the data sources on how to export data: specifically, it tells them if they should perform basic or extended export. If several applications request extended data collection, it is also the DCC's responsibility to determine when basic data collection can resume and to instruct the data sources to do so. The DCC works in conjunction with data collectors dedicated to specific data sources and implements the configuration of these according to the interfaces and capabilities provided by data sources.

A use case example where the DCC is involved for extended data collection may look something like this:

1. An application interrogates the catalog and discovers that extended data collection from data source 1 is possible but deactivated.
2. The application requires access to the extended set of data and requests that the DCC initiate extended collection through the data collector for data source 1.
3. The DCC supervises the extended reporting.
4. The DCC switches off the extended data collection when no applications require it.

The data relay gateway

The DRG provides functionality to transfer data from a customer network to an external AC in accordance with customer contracts and legislation.

It provides a generic solution for any collected data in a customer network and works in harmony with applications in the customer network that require access to the same data.

The data ingestion relay functionality in the DRG functional entity is placed in the customer network and in an external AC. These are named DRG-CN (customer network) and DRG-AC (application cluster). DRG-CN implements data export policies in accordance with customer contracts and regulations, and it transfers data to DRG-AC, which is the entrance to the ACs. DRG-CN is installed per customer and DRG-AC can be installed per AC or be shared by several ACs.

The solution centralizes the data ingestion relay functionality into one scalable functional entity (DRG-CN) that implements data export policies for any data transferred from customer networks to an external AC (DRG-AC). This avoids duplication of this functionality in applications that collect data in customer networks when the same data should be exported to external ACs.

The DRG-CN connects to the EDCA to access both streaming data and files as soon as these are available. In fact, the DRG behaves like any EDCA application in the customer network but with a single purpose: to transfer data to external ACs where it can be used for many purposes, such as training AI/ML models, serviceability, analytics and proactive product support, and so on. Together, the DRG-CN and DRG-AC form a trusted and secure mechanism to exchange data where, for example, one secure VPN tunnel can be used for all data collected by data collectors in customer networks, and for applications in an AC.

The DRG-CN also connects to a database that stores customer contracts according to a defined information model that expresses the written contracts in a formal language that can be used to execute policies in the DRG. The DRG only transfers what it is allowed to transfer according to the contracts, and when it performs a transfer it does so in the manner specified by the relevant contract(s). This means that some data elements must be de-identified with the help of data refinement

functionality before they can leave the customer networks. Examples of such data elements include customer identities such as IMSI (the International Mobile Subscriber Identity) and MSISDN (the Mobile Subscriber Integrated Services Digital Network).

Data distribution central functionality

DDC functionality facilitates cooperation between ACs, enabling them to discover and share data, thereby forming a virtual federated data lake. DDC procedures can be grouped into two sets: one with procedures for exchanging metadata, and the other with procedures for transferring data. The first set of procedures is used to discover data, specify requested data and receive notifications of updated data. The second set is used for data transfer and specifies how data sets can be exchanged with details needed for such interactions (with SFTP (SSH File Transfer Protocol) according to schema X, addresses, ports and so on). The DDC procedures enable ACs to automatically discover and consume data that exists in other ACs, as long as the customer contracts allow it.

The global data catalog and federated data lake

The GDC contains all the data in all of the ACs, including metadata regarding orders, customers, products and customer networks. Together with the DDC functionalities of all the ACs, the GDC creates a globally federated data lake. The GDC also serves as the root for discovering data available in the data lake.

Access to all subsets of data will be governed by policies based on contractual rights, with the full set of data visible only to applications and people with the right security classification.

THE DRG ONLY TRANSFERS WHAT IT IS ALLOWED TO TRANSFER ACCORDING TO THE CONTRACTS

Helper applications for data refinement

Data refinement functionality can be implemented with a set of helper applications that process the raw data available on the EDCA. The purpose of having helper applications is to provide supporting functionality for the applications that realize use cases. The helper applications for data refinement shown in Figure 2 realize data pipeline functionality for data storage, data processing, data serving, data visualization and data exposure.

Other examples of helper applications include:

- » security helper applications for de-identification of public data and non-public data
- » ETL (extract, transform, load) helper applications for the transformation and preprocessing of raw data into filtered and aggregated data sets
- » other repositories to persist structured data for data consumers (applications) with certain needs, such as graph databases for knowledge management.

Helper applications have the ability to offer refined data back to the EDCA for consumers to access it – for example, to applications realizing use cases (CEM, for example) or for exposure in service exposure frameworks. Further, applications that realize use cases and process the refined data to produce insights can also publish those insights in the EDCA for consumption by other applications realizing different use cases. The access rights to the insights and any other data set in the EDCA will be controlled with proper security mechanisms for authentication and policy-based authorization.

Conclusion

Modern intelligent applications depend on the availability of large amounts of high-quality data. In the telecom industry, where many of them rely on the same data sets, gathering the same data for different purposes is a waste of time and resources. In an increasingly data-driven world, it does not make sense to lock data into a single data pipeline or application. On the contrary, collected data should

COLLECTED DATA SHOULD BE AVAILABLE FOR USE IN THE DATA PIPELINE OF ANY APPLICATION SUITE THAT NEEDS IT

be available for use in the data pipeline of any application suite that needs it. At Ericsson, this is what we mean by democratizing data.

Our research shows that by harmonizing the data ingestion architecture of data pipelines, it is possible to ensure that the necessary data for all applications, regardless of where they are deployed, is only collected once. We can then make the data available to any application that needs it with the help of a data relay gateway. This secure and efficient solution provides the significant benefit of freeing up application development resources to focus on use-case realizations rather than data management.

THE AUTHORS



Anna-Karin Rönnberg

◆ is an AI and data expert with broad experience ranging from systems management to portfolio management. She joined Ericsson in 1985 and currently serves as a portfolio manager within the CTO office. In recent years, Rönnberg's work has focused on enabling a more data- and AI-driven strategy and portfolio. She holds an M.Sc. in electrical engineering from KTH Royal Institute of Technology in Stockholm, Sweden.



Bo Åström

◆ is an expert in system and service architectures within Business Area Networks. He joined Ericsson in 1985 and has extensive experience in radio networks, core networks and enterprise service networks, where he has worked with standardization and product development. Earlier in his career, Åström held technology specialist roles in the areas of interfaces and protocols, messaging architectures and network architectures. He holds more than 70 patents.



Bulent Gecer

◆ joined Ericsson in 2001. He is an expert in data management and incident analytics within Ericsson's CTO office and has played a leading role in developing data-management- and analytics-related architectures at corporate level for the past 10 years. Gecer previously worked as a technology specialist and software developer within core networks. He holds an M.Sc. in engineering physics from Uppsala University in Sweden.

Further reading

- » Ericsson, Autonomous networks, available at: <https://www.ericsson.com/en/future-technologies/autonomous-networks>
- » Ericsson, Future network security, available at: <https://www.ericsson.com/en/future-technologies/future-network-security>

Artificial intelligence in RAN

A SOFTWARE FRAMEWORK FOR AI-DRIVEN RAN AUTOMATION

Artificial intelligence and its subfield machine learning offer well-established techniques for solving historically difficult multi-parameterization problems. Used correctly, these techniques have tremendous potential to overcome complex cross-domain automation challenges in radio networks. Our ongoing research reveals that an integrated framework of software enablers will be essential to success.

DIARMUID CORCORAN,
ANDREAS ERMEDAHL,
CATRIN GRANBOM

Modern telecommunications and mobile networks are becoming increasingly complex from a resource management perspective, with diverse combinations of software and infrastructure elements that need to be configured and tuned for efficient operation with high QoS.

■ The latest 5G mobile system is a good example of a sophisticated radio network that allows many deployment variations – such as centralized, distributed or various hybrids of both – while simultaneously supporting diverse categories of applications such as mission-critical control with ultra-reliability and low latency, massively

concurrent Internet of Things device access and enhanced mobile broadband. It is well accepted in the communications community that appropriately dimensioned, efficient and reliable configurations of systems like 5G are a complex technical challenge. Increased real-time, closed-loop, automation at all levels and time frames is a critical tool in controlling this complexity and ultimately reducing the capex and opex of radio network operations. There is currently a significant trend toward the use of artificial intelligence (AI) technology as an enabler for automating both repetitive and complex tasks. Applied correctly, AI and machine learning (ML) techniques have the potential to boost system

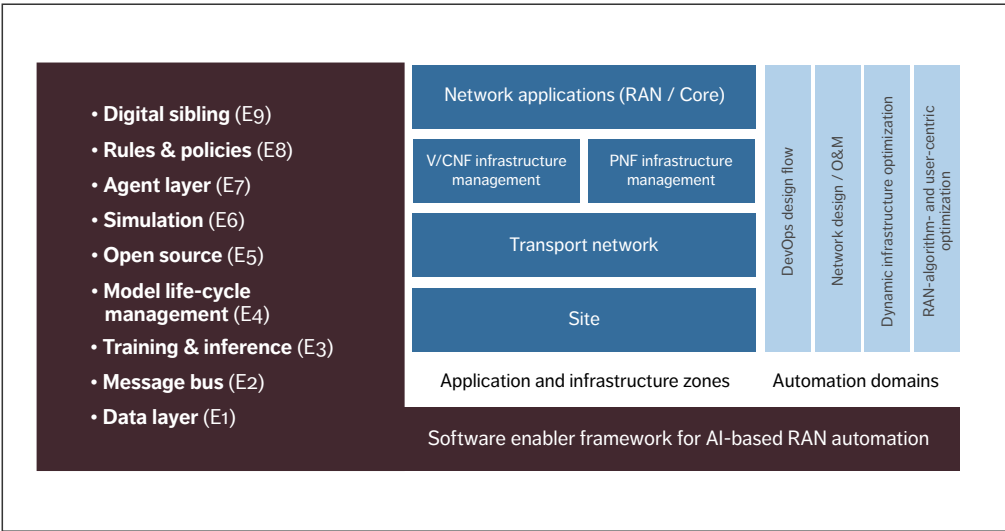


Figure 1 Software-enabler framework for AI-based RAN automation

efficiency by drastically simplifying certain types of automated management and control. It is important to realize, however, that this development will not occur without effort; on the contrary, the need for AI and ML techniques will have a profound effect on how the next generation of RAN should be designed and realized.

As part of this work, we have created a set of software enablers that target efficient and effective use of AI technologies in RANs. Together, they form an important framework of building blocks in realizing the next level of autonomous, AI-based, resource management in 5G and future-generation networks.

Landscape of automation in RAN

The wide variety of architectural and transport options that are possible in 5G results in several different configuration permutations [1]. RAN application(s) can be distributed in a flexible way over both physical (PNF) and virtualized/container (V/CNF) infrastructure, as shown in the middle section of *Figure 1*. How RAN application

parameters are chosen and tuned is partially dependent on the selected distribution over compute, transport and site resources. The right side of *Figure 1* shows the four automation domains that play a role in optimal network tuning across applications and infrastructure:

- » DevOps (development and operations) design flow
- » Network design/operations and maintenance (O&M)
- » Dynamic infrastructure optimization
- » RAN-algorithm and user-centric optimization.

Sophisticated use of AI techniques involves deep and disruptive changes to many existing software engineering, algorithm design and workflow stages. The DevOps (Development and Operations) flow, for example, needs to be extended with new data and ML model life-cycle management (MLCM) procedures. Network design/O&M, dynamic infrastructure optimization and RAN-algorithm/user optimization also need to be reengineered in a radically but carefully staged approach [2].

Software-enabler framework for AI-based RAN automation

Our research and supporting proof-of-concept systems have shown that a set of software enablers is required to best facilitate new AI approaches to sophisticated closed-loop automated network design. These enablers must be tightly integrated into a software framework that allows flexible and open information exchange. As shown on the left side of Figure 1, our software-enabler framework for AI-based RAN automation consists of nine key enablers:

- » A data layer (E1)
- » A message bus (E2)
- » Training and inference (E3)
- » Model life-cycle management (E4)
- » Open-source AI software (E5)
- » Integrated simulation (E6)
- » A decision-making agent layer (E7)
- » Rules and policies for intent-based management (E8)
- » A digital sibling for knowledge harvesting (E9).

Perhaps the most essential enabler of all is the production of industry-standard, machine-readable data (E1) at a high temporal rate by RAN network nodes (eNodeB or gNodeB). This data can be aggregated, stored, filtered and distributed by advanced real-time, hierarchical, message bus (E2) solutions. A distributed training and inference (E3) architecture can efficiently consume this data and use it to facilitate the training of ML models to create new AI algorithms and/or parameter selection for classic algorithm design [2].

There are many alternatives [3] to creating/ training models for use with ML algorithms, with three main classifications:

1. Supervised learning, in which algorithms are trained using labeled examples
2. Unsupervised learning, using a cluster or grouping technique
3. Reinforcement learning (RL) [4], using a target reward strategy that allows guidance toward an optimal set of actions.

Depending on the problem targeted, the best ML model and supporting life cycle (E4) can vary considerably in complexity. For simpler problems, linear regression, smaller decision-tree and simple neural networks (NNs) with a few nodes and layers can be sufficient. For more complex problems, large decision trees or deep neural networks (DNNs) with many layers and nodes and several convolutional layers could be necessary to achieve the required accuracy.

RL approaches and supporting agents (E7) for controlling optimization goals can be especially effective at learning novel RAN management strategies. Training RL models depends on active exploration, through a software agent’s trial and error experiments, which will not always be possible or appropriate in a live RAN system. To help solve this problem, and to generate the required quantities of data to train models, we have included simulation (E6) in our set of software enablers.

Once trained, an ML model can be used in the inference phase (part of E3), where a selection of data is used as input into the model that will then produce a set of predictions, actions or rules, with the exact details depending on the ML algorithm type. In RAN, the hardware and software requirements on the training and inference phases can be vastly different. Training typically requires powerful central processing unit or specialized graphics processing unit (GPU) hardware with large memory and data storage.

AI software platforms, such as TensorFlow, Keras and PyTorch, together with other extensive, open source (E5), often Python-based, ML software ecosystems need to be integrated into the software engineering flow. During the inference phase, a trained model (or models) is made available to the RAN application(s) through model life-cycle management (E4). For latency-critical RAN edge applications, the inference needs to be efficiently realized, with low latency, power consumption and memory footprint, taking the characteristics of the target hardware and software architecture into account.

Our software enablers are fully compatible with intent-based management solutions [5].

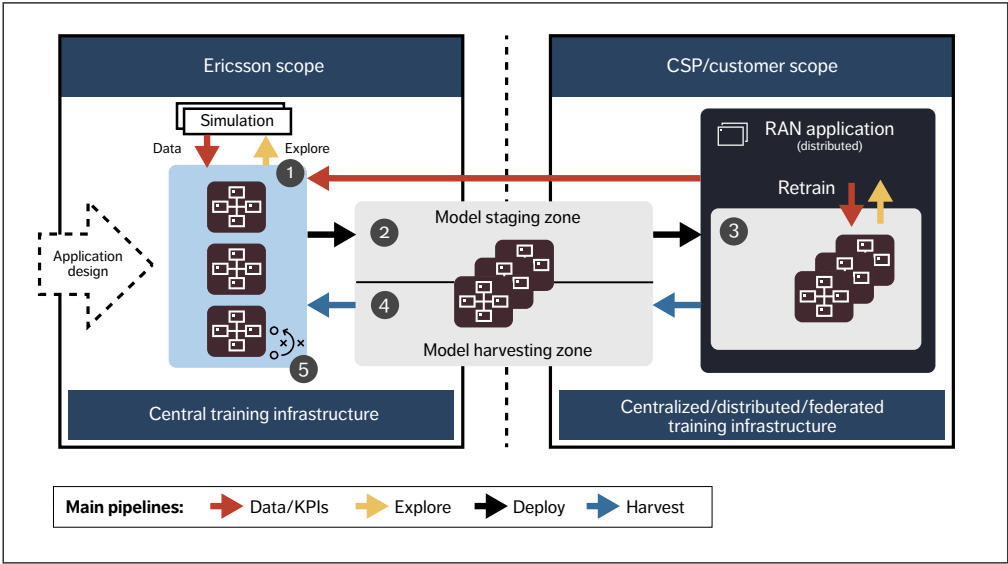


Figure 2 Model life cycle management

When moving toward AI-based automation, operators will no longer spend time tweaking RAN performance by adjusting individual parameters exposed by the RAN system. Instead, well-defined high-level rules and policies (E8) for expressing overall operational intent are used. The AI-based automation machinery is then responsible for realizing these goals.

Long term, to allow the most sophisticated forms of automation, we see a need to build partial digital representations of deployed systems. Our unique digital sibling (E9) concept makes it possible to harvest ML models to build context-specific knowledge.

Model life-cycle management

Machine-learning models are valuable, evolving assets. As such, a sophisticated and controlled software engineering chain (E4) is required for MLCM. When a model is updated through retraining outside its initial context, for example, many significant challenges are introduced in terms of traceability, stability and the possibility to identify

faults. With potentially many instances of a model deployed at cell, cell cluster or indeed user level, this challenge is compounded. Figure 2 captures the main conceptual stages in model management from a software process perspective.

After an initial model design and training phase (marked as 1 in Figure 2), possibly supported by simulation components, the model enters a staging zone (phase 2). This is where customer- or hardware-specific model adaptations [6] occur and are packaged into an industry-deployment format. For example, a DNN can be pruned to minimize nodes that are not significant with additional quantization transformation to reduce the number of bits required to represent a number. This can significantly reduce size, producing an embedded DNN device representation with just a minor impact on accuracy. In phase 3, the model(s) is deployed into a RAN application for use. Initially, these models will be tightly integrated with the RAN application. In the long term, however, it will be necessary to separate model and application life cycles to allow for rapid model training/retraining and deployment cycles.

WE ENVISION THE USE OF OUR NOVEL MODEL-HARVESTING TECHNIQUE TO CREATE DIGITAL SIBLINGS

Retraining and model drift

Certain types of models (those dependent on traffic and variation in load, for example) may not initially perform well in their deployed context. Over time, there may also be drift between the model training context and the current operating state. As such, it is necessary to allow for periodic, active retraining with context-specific data and features in phase 3.

Model harvesting

Model harvesting is a way to improve simulation and ML training capabilities by enabling a model feedback loop from deployment to design contexts. Phase 4 includes the anonymized collection of model data from a CSP deployment back into our model harvest zone. This facilitates sophisticated data mining from these models, including valuable clustering information on RAN resource usage. The harvested data can then be used as input for model training and retuning when the simulator configurations are updated, for example. In the longer term we envision the use of our novel model-harvesting technique to create digital siblings (phase 5) that partially reflect different real-world RAN deployments.

Test, verification and run-time monitoring of RAN applications utilizing machine learning

A data-driven and continuously learning RAN will introduce new requirements on how software is tested, verified and monitored during system run time. Today, exhaustive test and verification is performed, both on individual software modules and at system level, automated over various continuous integration loops before deployment. The same software is deployed on most RAN nodes, perhaps somewhat differentiated depending on the hardware, surrounding equipment and customer preferences. However,

when moving toward RAN systems that utilize ML and RL technologies, this will no longer hold.

For example, ML models may develop independently and not be the same for all RAN nodes, and they may not remain constant between RAN software releases. As a result, new ways to continuously test and validate RAN software utilizing ML models must be developed to fit the MLCM phases.

Moreover, new fine-grained KPIs and metrics must be collected during system run time and fed back to the MLCM. This includes new KPIs for deployed ML models, such as their perceived accuracy, as well as performance and resource consumption KPIs for the RAN applications utilizing the ML models. Similar to other type of RAN data relevant for machine learning, the KPIs should be distributed by the data layer.

A data layer targeting machine learning in the RAN

A cornerstone for producing high-quality ML and RL models is good access to data. Currently, there are many mechanisms for the generation, collection and analysis of data from the RAN. One example is the performance management and configuration management data reported in a standardized format from RAN nodes. Another example is debug and trace data, which includes detailed information about the internal RAN system state. The latter data is often generated on demand – that is, associated with a trouble report or customer service requests.

When targeting generation and use of ML models for time-critical RAN control loops, none of the existing mechanisms is a perfect match. At Ericsson, we are actively investigating and introducing new mechanisms for the generation, management and distribution of data suitable for ML in the RAN. Our lightweight data generation mechanisms allow high quantities of data to be generated without adversely affecting core RAN system performance.

The data is produced using a few standardized machine-interpretable formats. Each data format has an associated schema allowing for easy serializing, deserializing and content interpretation. The data

management allows different data sources to be controlled and activated in a very fine-granular manner. Information on the data and associated schemas is stored and organized in distributed databases, called data- and schema inventories.

Our data distribution is based on message bus technologies, allowing the produced data to be automatically tagged with relevant keys and to be automatically routed to different subscribers based on their tag subscription requests. Our research-developed message bus is hierarchical and distributed, meaning that the produced data will not be routed longer than needed, making it particularly suitable for the requirements of a virtualized and disaggregated RAN.

Enhancing machine learning in the RAN with simulation

Despite ongoing work to improve high-fidelity data collection in the RAN, its distributed nature means it is likely that data will continue to be sparse and that data collection will continue to be a resource-consuming task. At the same time, learning through exploration can be difficult in live systems, where exploration may degrade or crash the system.

Our approach to mitigating these challenges is to integrate in-loop simulation (E6) into our framework, providing the means for much richer data generation as well as the use of explorative RL techniques. Allowing in-loop simulation creates new possibilities for complex, multistep, scenario exploration without compromising system integrity. The first phase of our in-loop simulation is restricted to the Ericsson development scope shown in Figure 2, but it will evolve to include customer deployment in the future.

Looking ahead, more sophisticated AI-based automation in the RAN will require partial digital representations of deployed systems to support complex decision-making. In our framework, we call this enabler a digital sibling (E9).

Digital sibling

The digital sibling is a way to capture and transfer data, and especially ML model data, from a real-world context. These digital representations can then be used for increasingly accurate data generation and in-loop simulation. A related concept, the digital twin, endeavors to duplicate, more completely, a full digital copy of a physical resource or system. The scope of our digital sibling is limited to the transferable digital knowledge, as captured in models that have been trained, interchangeably, in both a simulated and real-world RAN environment.

After an initial training phase, an ML model captures a considerable amount of transferable, problem-specific knowledge, including knowledge that is compressed and coded in the weights of an NN or a set of rules in a decision tree. Sets of models can be used to capture the different decision-making needs of a system. For example, a set of ML models can be trained in a local CSP context to predict the best link adaption or power control parameters for a user.

Other examples include the choice of geographical baseband workload placement required for a given service category and anomaly detection like rogue users or a base station with malicious intent. This aggregated model information (transferred through our MLCM in phase 4), together with actions by goal-oriented software agents, forms a considerable body of knowledge that

Terms and abbreviations

AI – Artificial Intelligence | CSP – Communication Service Provider | DevOps – Development and Operations | DNN – Deep Neural Network | DSP – Digital Signal Processor | GPU – Graphics Processing Unit | ML – Machine Learning | MLCM – Model Life-Cycle Management | NN – Neural Network | PNF – Physical Network Function | RL – Reinforcement Learning | V/CNF – Virtualized/Container Network Function

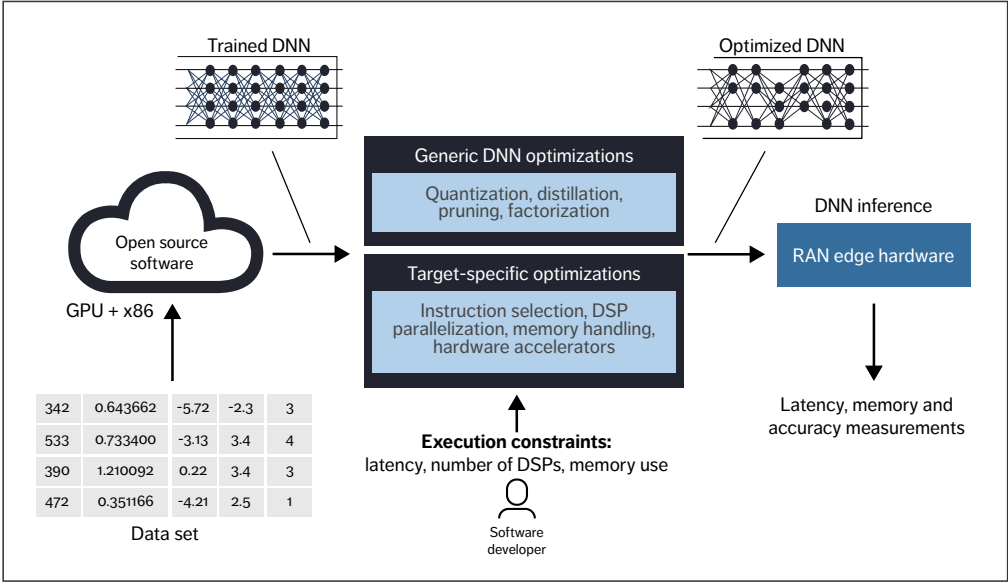


Figure 3 DNN inference optimization for RAN edge hardware

can continuously evolve to form a learning system. The digital sibling is a unique part of our framework, which will ultimately enable advanced automation.

Proof of concept #1 – deep neural network inference optimization

In the proof of concept shown in *Figure 3*, we have investigated how to tune, optimize and realize pretrained DNNs for inference on RAN edge hardware. We selected this example because radio- and baseband data processing are time-critical and computationally intensive tasks. These steps would be part of phase 2 in *Figure 2*.

The DNN training was done in a container-based environment (Kubernetes) using GPUs and state-of-the-art open source DNN training frameworks. Several generic DNN optimization techniques were investigated and implemented, such as quantization, distillation and pruning, with the goal to reduce the footprint and execution time of the DNN inference, still keeping the accuracy close to the accuracy of the originally trained DNN.

The optimized DNN was realized in terms of digital signal processor-C (DSP-C) code – that is, C extended with some specific DSP keywords and instructions, taking into account constraints given by the target RAN hardware and the RAN software designers, such as the number of DSPs and the amount of memory available for the DNN inference. Latency and memory measurements were made on the RAN edge hardware platform for the generated DNN DSP-C code. We also tuned how to best parallelize the DNN inference over several DSPs.

Based on performed evaluations, we concluded that DNN inference for the target hardware can benefit significantly from the generic- and target-specific DNN optimizations implemented. However, there is normally a trade-off between ML model accuracy, execution time and memory consumption.

Proof of concept #2 – AI-accelerated RAN platform

The proof-of-concept platform shown in *Figure 4*

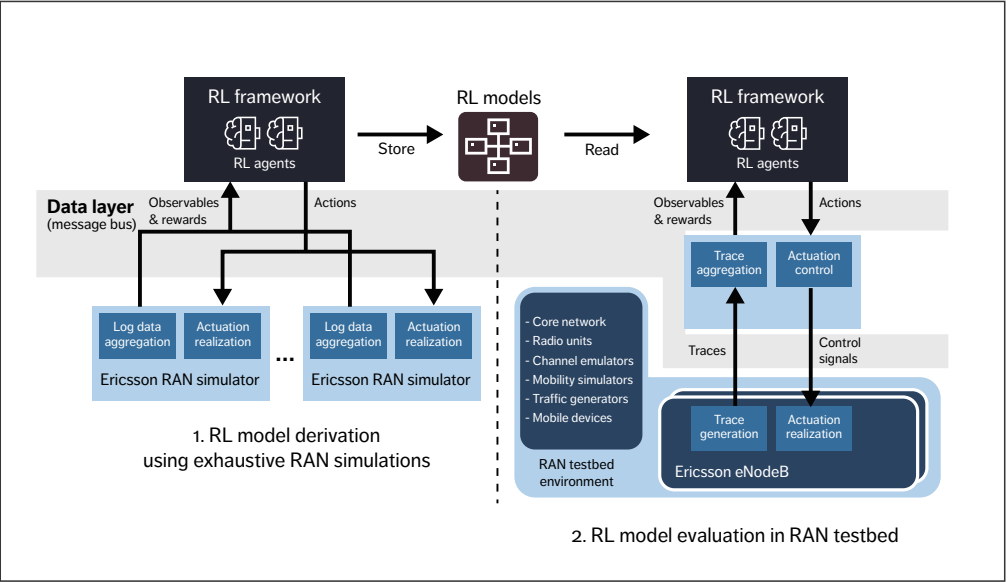


Figure 4 AI-accelerated RAN platform

supports the use of RL techniques and model transfer to control parts of the RAN functionality. Our second proof of concept explores phases 1, 2 and 3 in *Figure 2*. We have used sophisticated RAN simulators to set up and run many different deployment scenarios to train the RL models. The RAN test bed we used consists of several cells and multiple UEs (user equipment). The data layer uses a message bus, which organizes, aggregates and filters data.

To obtain a high degree of scalability and improve model generalization and RL exploration possibilities, the RAN simulators have been containerized, allowing many concurrent instances to execute with different configurations.

The RAN simulator can extract observables and rewards based on internal simulator data, as well as change its internal state and resource allocation strategies based on the actions provided by the RL agents. The RL agent framework is separated from both simulated and actual RAN application by the same well-defined data and action interfaces. RL algorithms can be developed and evaluated in

isolation from the RAN application, using RL domain experts, and with required compute and memory resources, including the use of powerful GPUs to train the DNNs that are used by the RL agents.

After achieving high performance in the different simulator scenarios, the simulator-trained RL models can be used for automation use cases in real-life RAN contexts.

Conclusion

Growing complexity and the need to solve repetitive tasks in 5G and future radio systems necessitate new automation solutions that take advantage of state-of-the-art artificial intelligence and machine learning techniques that boost system efficiency. At Ericsson, we have identified the key software enablers for AI-based RAN automation and integrated them into a comprehensive framework that provides a solid and flexible technological foundation for the development of AI-based RAN. Our two proof-of-concept examples, executed in real RAN contexts, clearly illustrate the benefits of our framework of software enablers in building the next generation of automated RAN systems.

References

1. Ericsson Technology Review, 5G New Radio RAN and transport choices that minimize TCO, November 7, 2019, Eriksson, A-C; Forsman, M; Ronkainen, H; Willars, P; Östberg, C., available at: <https://www.ericsson.com/en/reports-and-papers/ericsson-technology-review/articles/5g-nr-ran-and-transport-choices-that-minimize-tco>
2. Ericsson Technology Review, Enhancing RAN performance with AI, January 20, 2020, Calabrese, F.D.; Frank, P; Ghadimi, E; Challita, U; Soldati, P., available at: <https://www.ericsson.com/en/reports-and-papers/ericsson-technology-review/articles/enhancing-ran-performance-with-ai>
3. MIT Press, Deep Learning, 2016, Goodfellow, I; Bengio, Y; Courville, A., available at: <https://www.deeplearningbook.org/>
4. MIT Press, Reinforcement Learning: An Introduction (second edition), 2018, Sutton, S; Barto, A., available at: <http://incompleteideas.net/book/RLbook2020.pdf>
5. Ericsson Technology Review, Cognitive technologies in network and business automation, June 28, 2018, Niemöller, J; Mokrushin, L., available at: <https://www.ericsson.com/en/reports-and-papers/ericsson-technology-review/articles/cognitive-technologies-in-network-and-business-automation>
6. KTH Royal Institute of Technology, Squeezing and Accelerating Neural Networks on Resource Constrained Hardware for Real Time Inference, 2018, Presutto, M., available at: <http://www.diva-portal.org/smash/get/diva2:1299467/FULLTEXT02.pdf>

Further reading

- » Ericsson, How will AI enable the switch to 5G?, available at: <https://www.ericsson.com/en/networks/offerings/network-services/ai-report>
- » Ericsson, Autonomous networks, available at: <https://www.ericsson.com/en/future-technologies/autonomous-networks>
- » Ericsson, Ericsson launches unique AI functionality to boost radio access networks, available at: <https://www.ericsson.com/en/news/2019/10/ericsson-ai-to-boost-ran>
- » Ericsson, AI by Design, available at: <https://www.ericsson.com/en/ai-and-automation>
- » DeepMind blog, AlphaGo Zero: Starting from scratch, available at: <https://deepmind.com/blog/article/alphago-zero-starting-scratch>

THE AUTHORS



Diarmuid Corcoran

◆ joined Ericsson in 1992. He currently works within the Networks division of Ericsson as an expert in software architecture, actively driving and participating in software-related activities across the company. Corcoran has contributed important software and system solutions to five generations of mobile technology, including 5G. His current research interests include methods, architectures and algorithms for enabling self-learning resource management solutions with a focus on advanced automation. Corcoran holds a B.Eng. in computer engineering from the University of Limerick, Ireland.

Andreas Ermedahl

◆ joined Ericsson in 2010. He works as a software researcher within the Networks division, where he is responsible for driving a software program research area targeting AI and ML in RAN. His research interests include static program analysis, optimization methods and software techniques for facilitating ML and AI in RAN. Ermedahl received a Ph.D. in computer systems from Uppsala University in Sweden and earned the title of docent in computer science at Mälardalen University in Sweden. He currently holds an adjunct professorship in computer science at KTH Royal Institute of Technology in Stockholm.



Catrin Granbom

◆ has worked within Ericsson's telecom and information technology area since 1987, most recently with 4G and 5G. In recent years, her focus has been on research close to implementation, including collaboration ties to academia supporting this. She is currently responsible for software technology research activities within the Networks division, including software implementation technology in areas such as cloud RAN, AI software in RAN, test and quality, high-performance software and software for cyber-security. Granbom holds an M.S. in computer science from Uppsala University.



